

k-vizinhos mais próximos

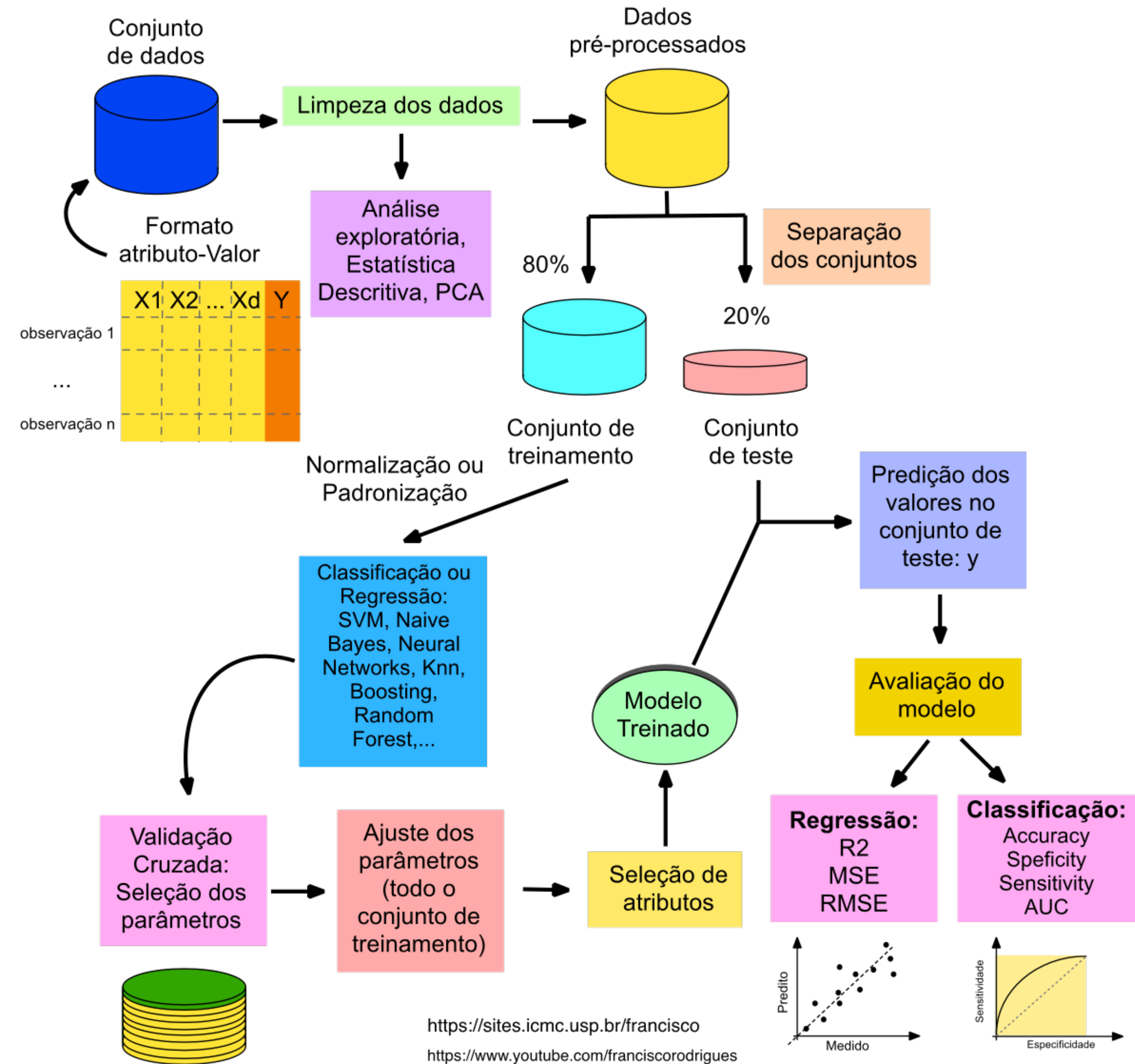
Francisco Rodrigues

Instituto de Ciências Matemáticas e de Computação

Universidade de São Paulo

francisco@icmc.usp.br

Onde estamos?



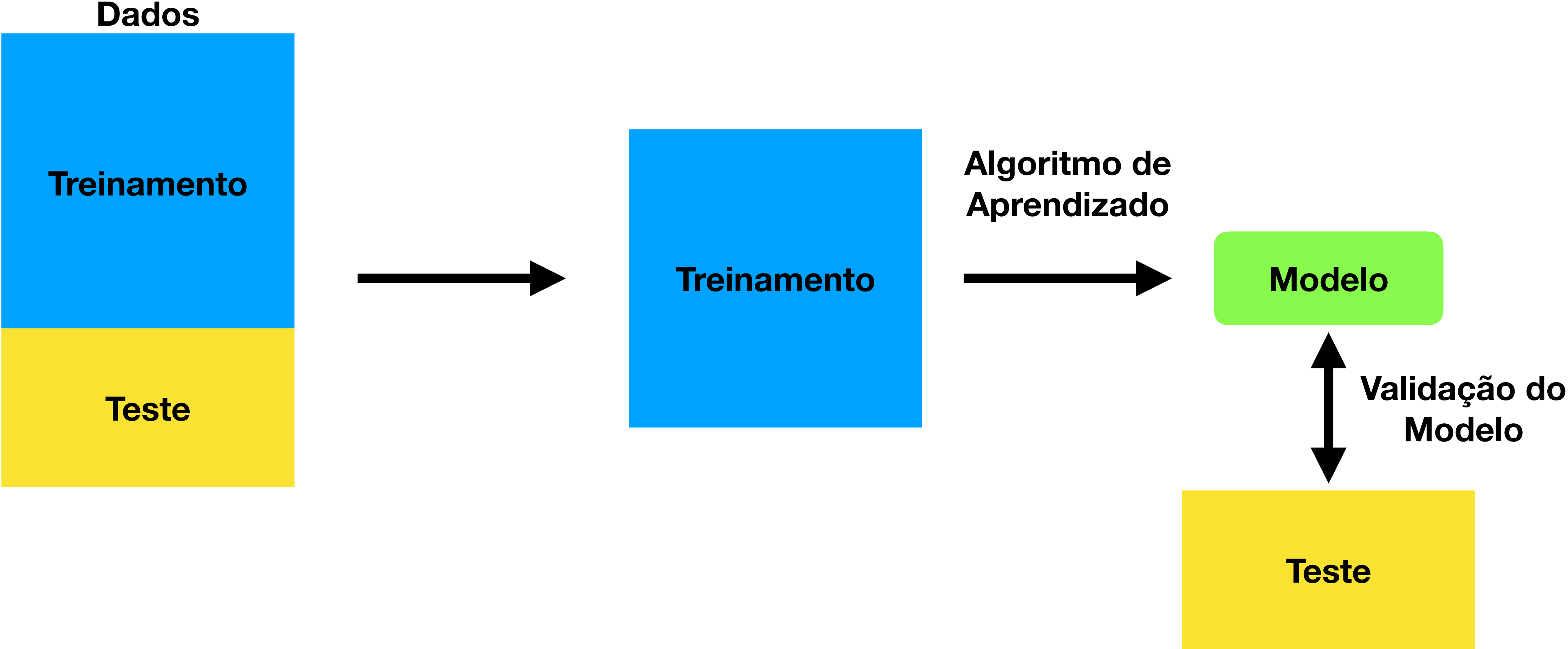
Método Holdout

Dados

Treinamento

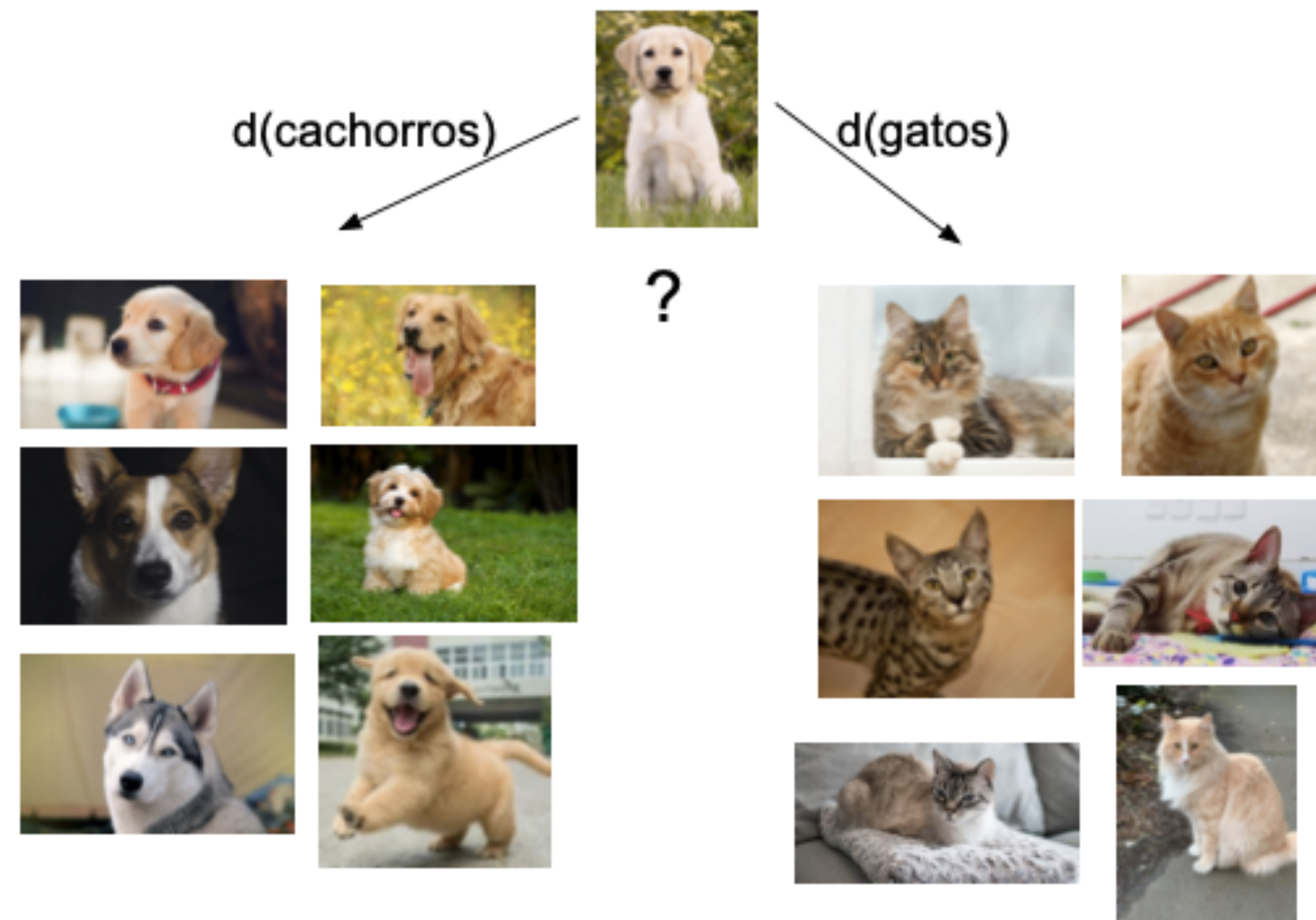
Teste

k-vizinhos



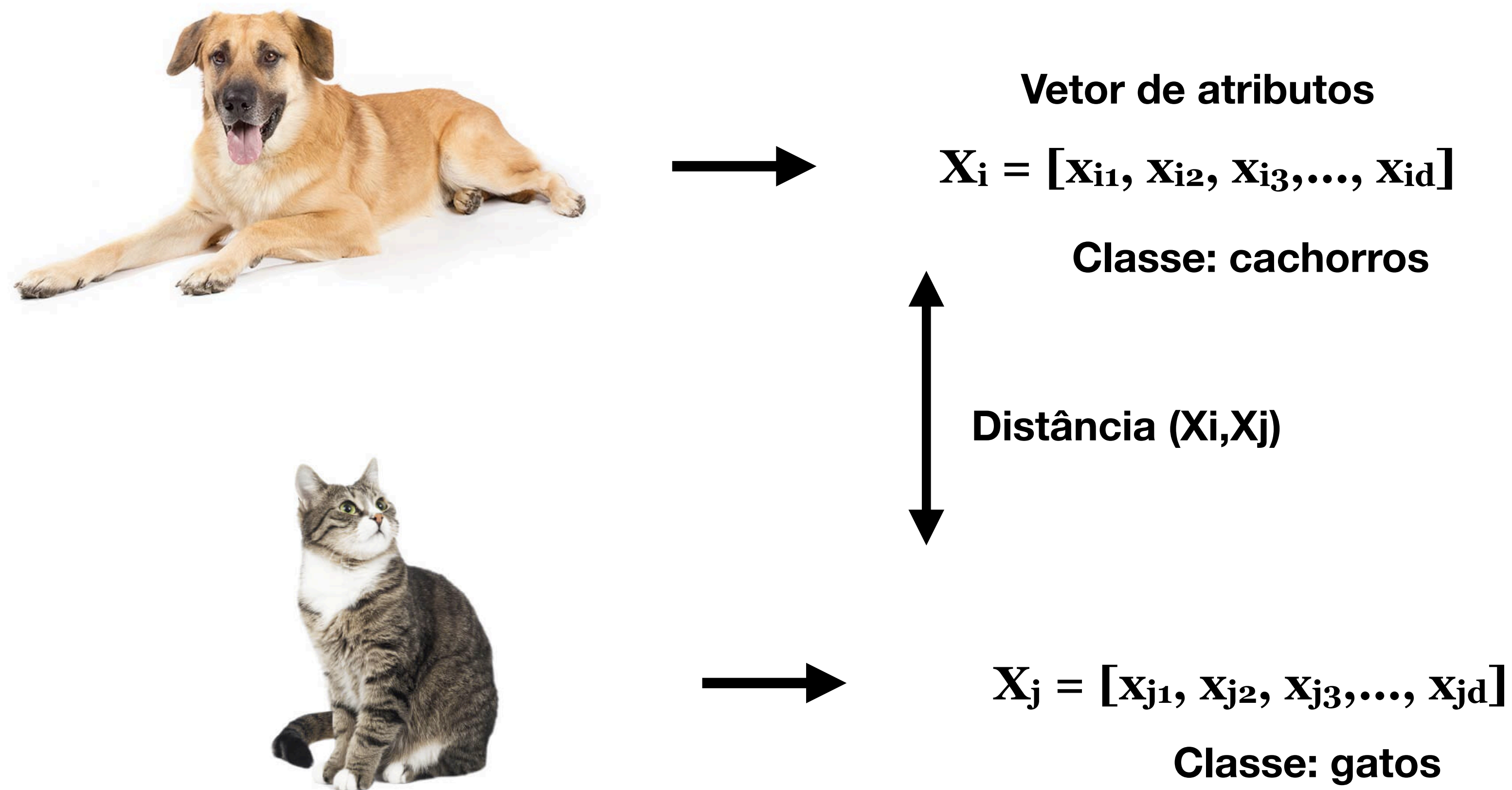
k-vizinhos

- Uma maneira simples de definimos a classificação de objetos é através de distância entre eles:



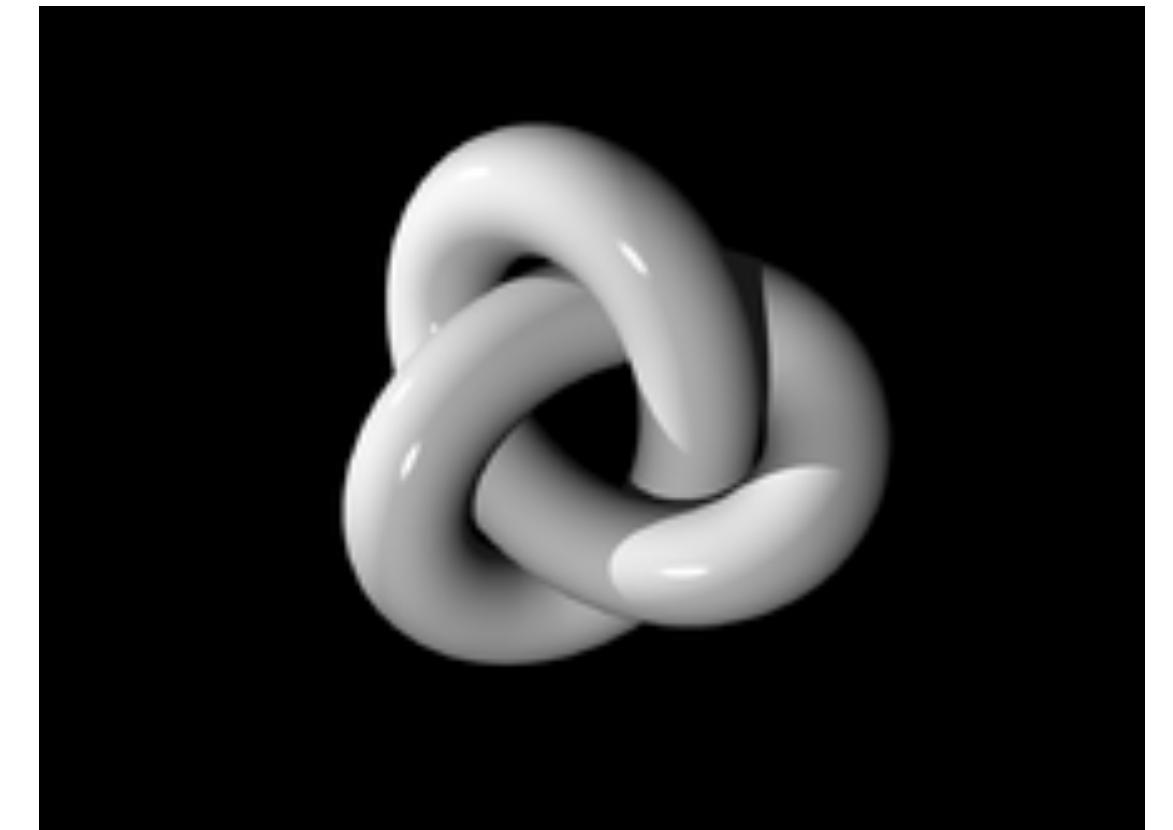
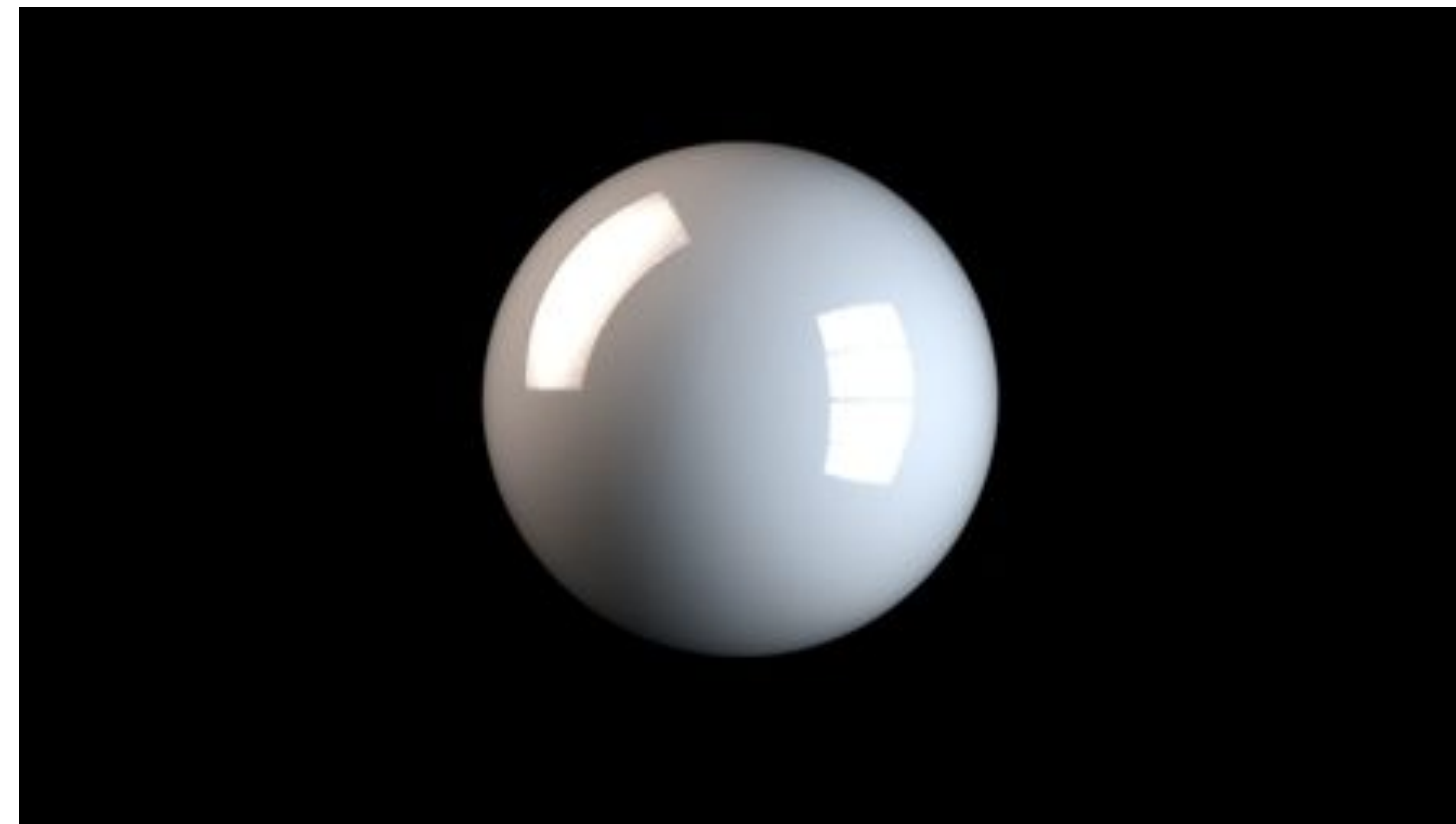
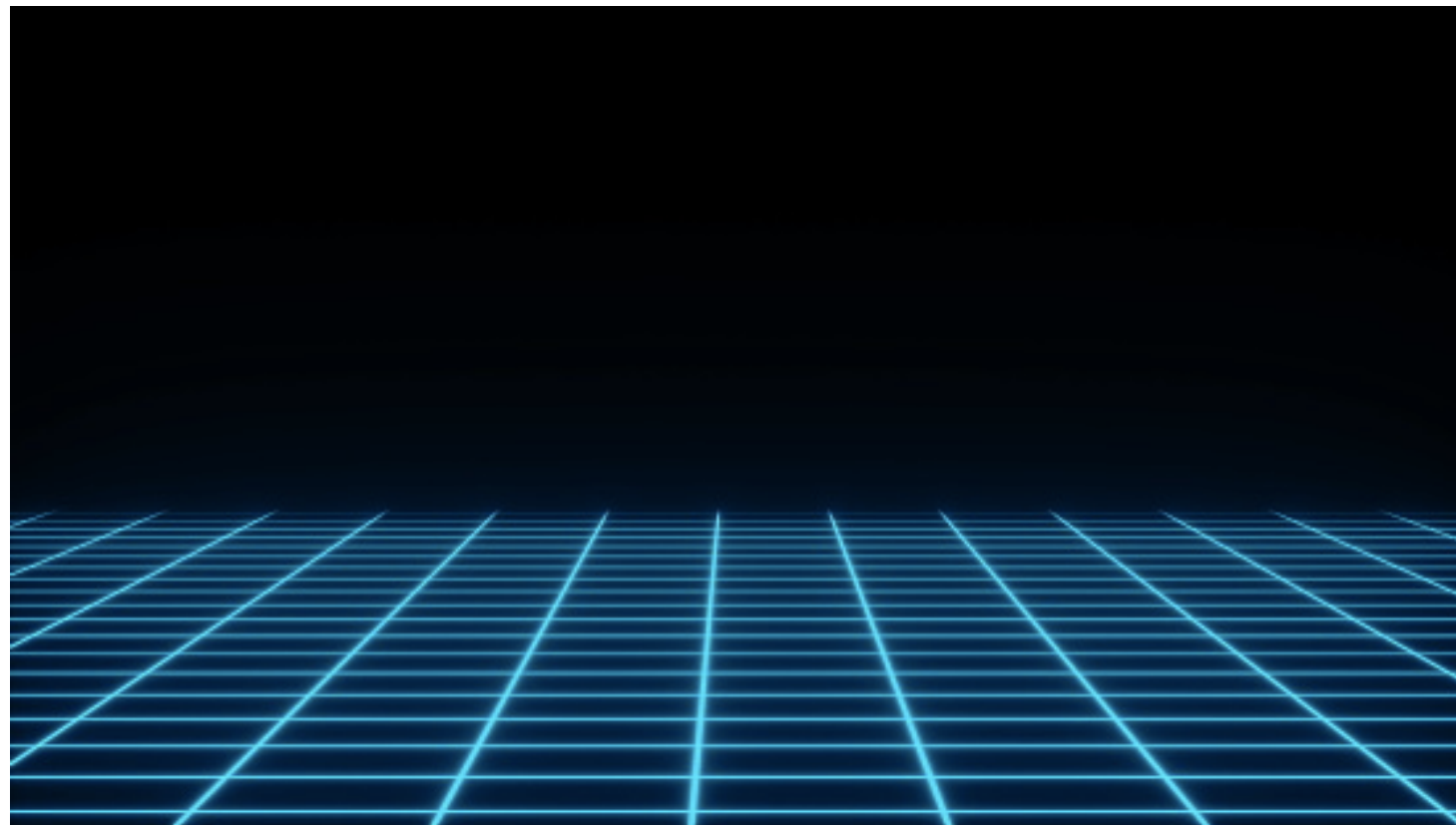
k-vizinhos

- Como definir a distância entre objetos?

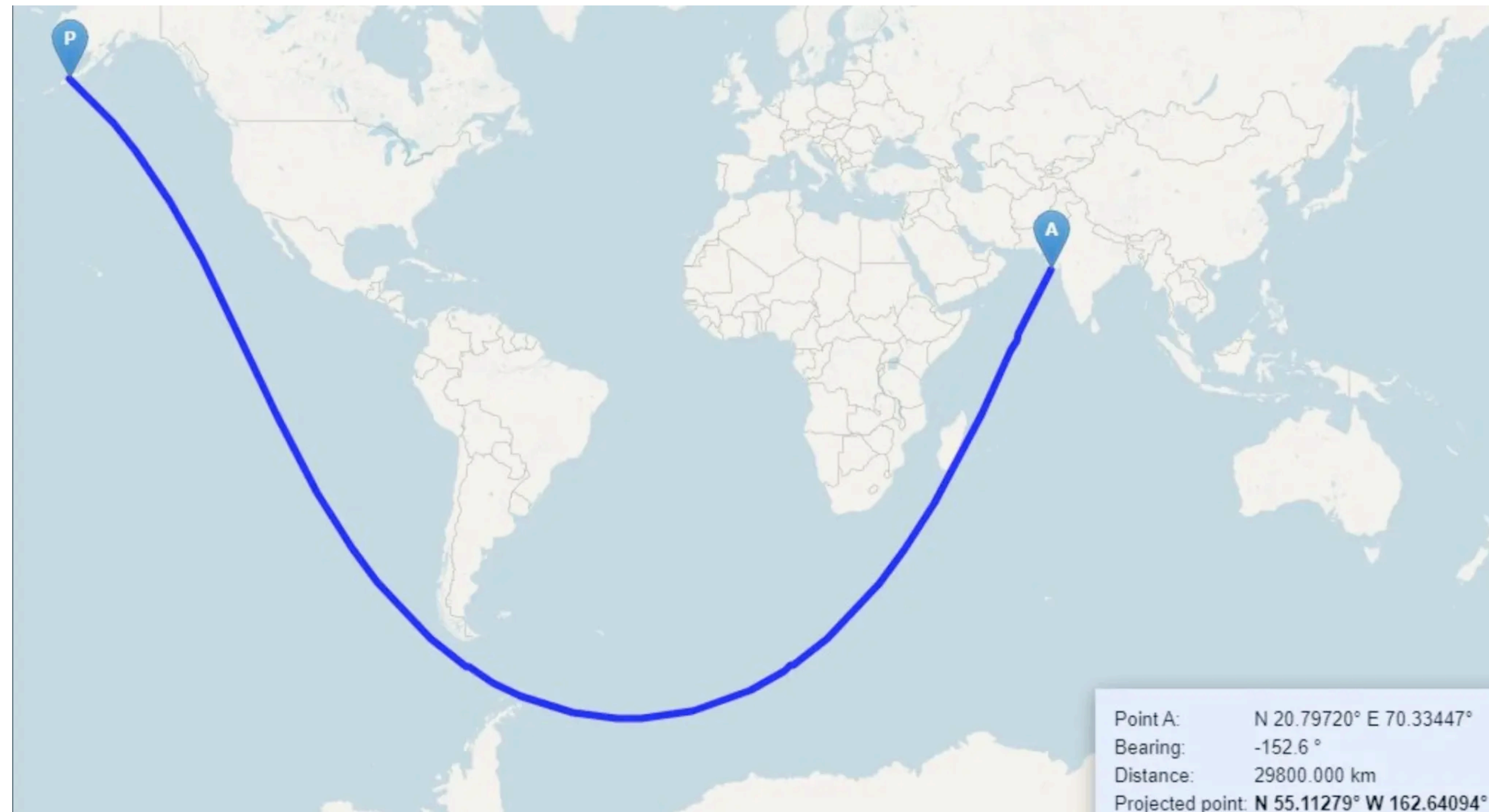


k-vizinhos

- Como definir a distância entre objetos?



k-vizinhos



<https://www.youtube.com/shorts/d2IP2ZKIQ9Y>

- Como definir a distância entre objetos?
- Precisamos definir uma medida de proximidade.
 - **Medida de similaridade:** $d(X_i, X_i)$ é máxima.
 - Exemplo: Número de amigos compartilhados em uma rede social.
 - **Medida de dissimilaridade:** $d(X_i, X_i) = 0$.
 - Exemplo: distância entre cidades (distância Euclidiana).

- **Medida de dissimilaridade:**

- $d(p, q) \geq 0$ para todo p e q , e $d(p, q) = 0$ se, e somente se, $p = q$,
- $d(p, q) = d(q, p)$ para todo p e q ,
- $d(p, r) \leq d(p, q) + d(q, r)$ para todo p, q , e r , onde $d(p, q)$ é a distância de dissimilaridade entre os pontos (objetos) p e q .

- **Medida de similaridade:**

- $s(p, q) = 1$ (ou máximo de similaridade) se $p = q$,
- $s(p, q) = s(q, p)$ para todo p e q , onde $s(p, q)$ é a similaridade entre os objetos p e q .

k-vizinhos

- Métricas de distância:

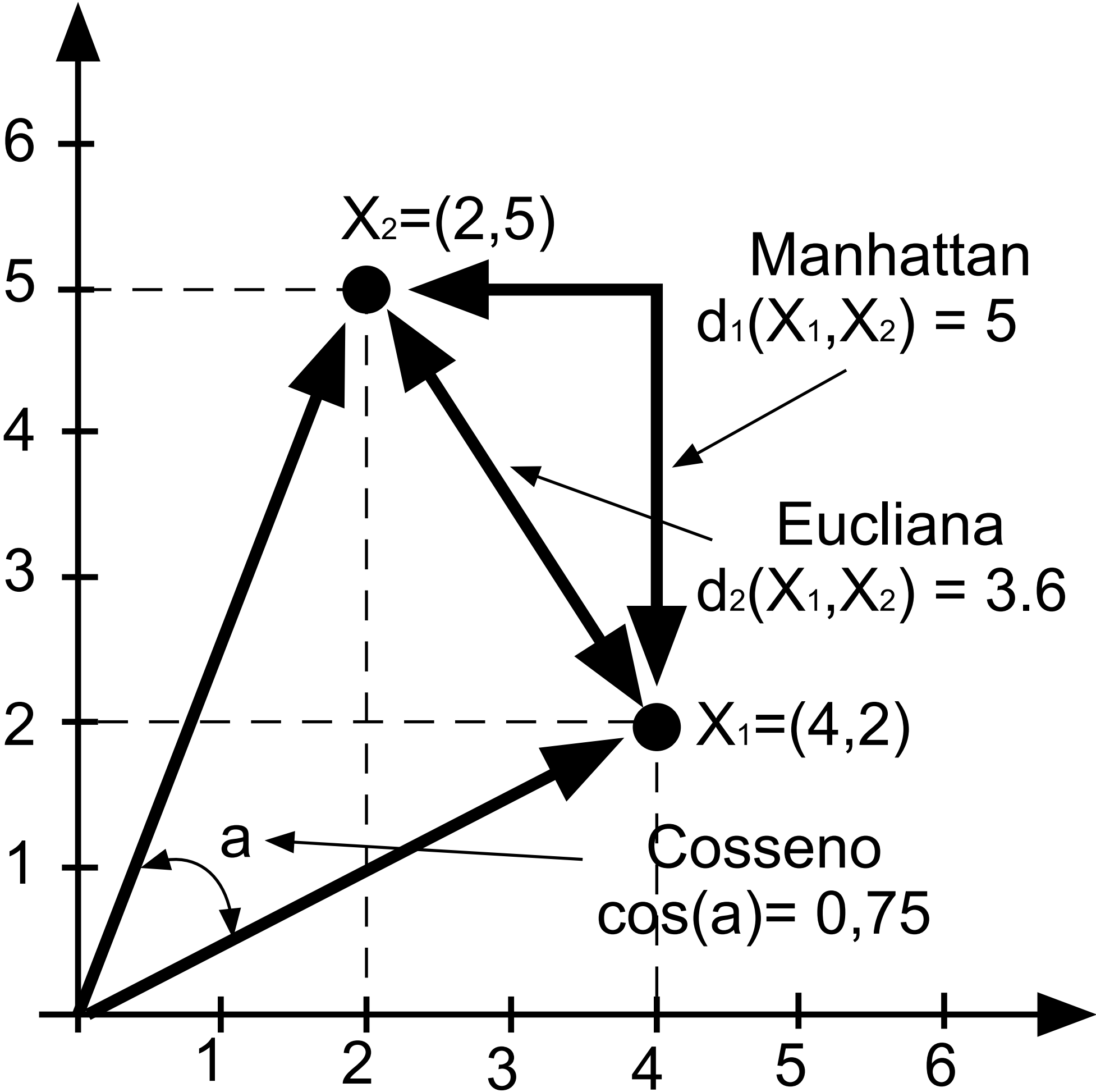
- Euclidiana $D(X, Y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad [0, \infty)$

- Minkowski $D(X, Y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{\frac{1}{p}} \quad [0, \infty)$

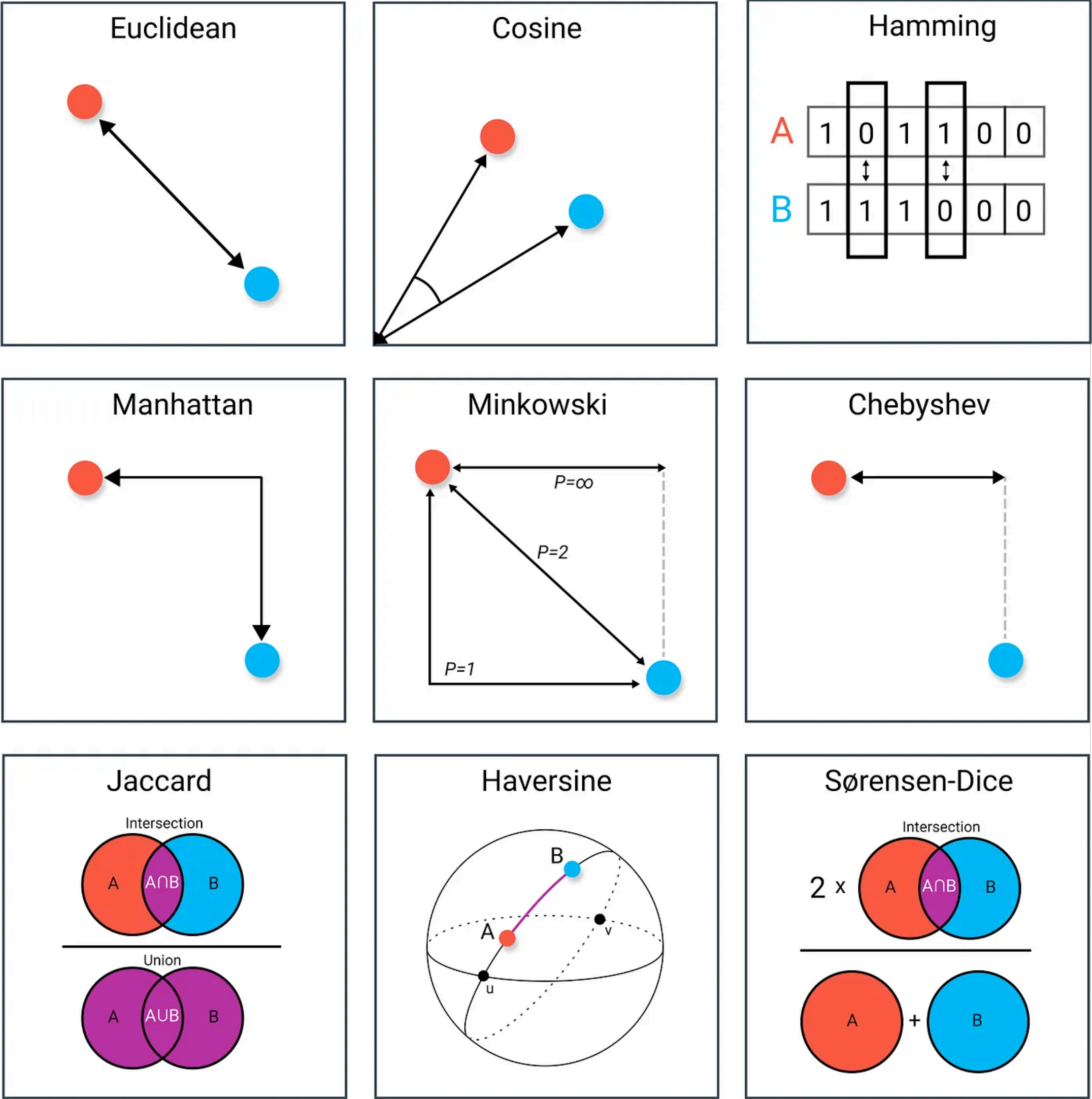
- Cosseno $D(X, Y) = \frac{\sum_{i=1}^n x_i y_i}{\sqrt{\sum_{i=1}^n (x_i)^2} \sqrt{\sum_{i=1}^n (y_i)^2}} \quad [0, 1]$

- Pearson $D(X, Y) = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \cdot \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} \quad [-1, 1]$

k-vizinhos



k-vizinhos



k-vizinhos

- Métricas de distância:

- Dados nominais

Similaridade

$$s = \begin{cases} 1 & p = q \\ 0 & p \neq q \end{cases}$$

Dissimilaridade

$$d = \begin{cases} 0 & p = q \\ 1 & p \neq q \end{cases}$$

- Dados ordinais

$$s = 1 - \frac{||p - q||}{n - 1}$$

$$d = \frac{||p - q||}{n - 1}$$

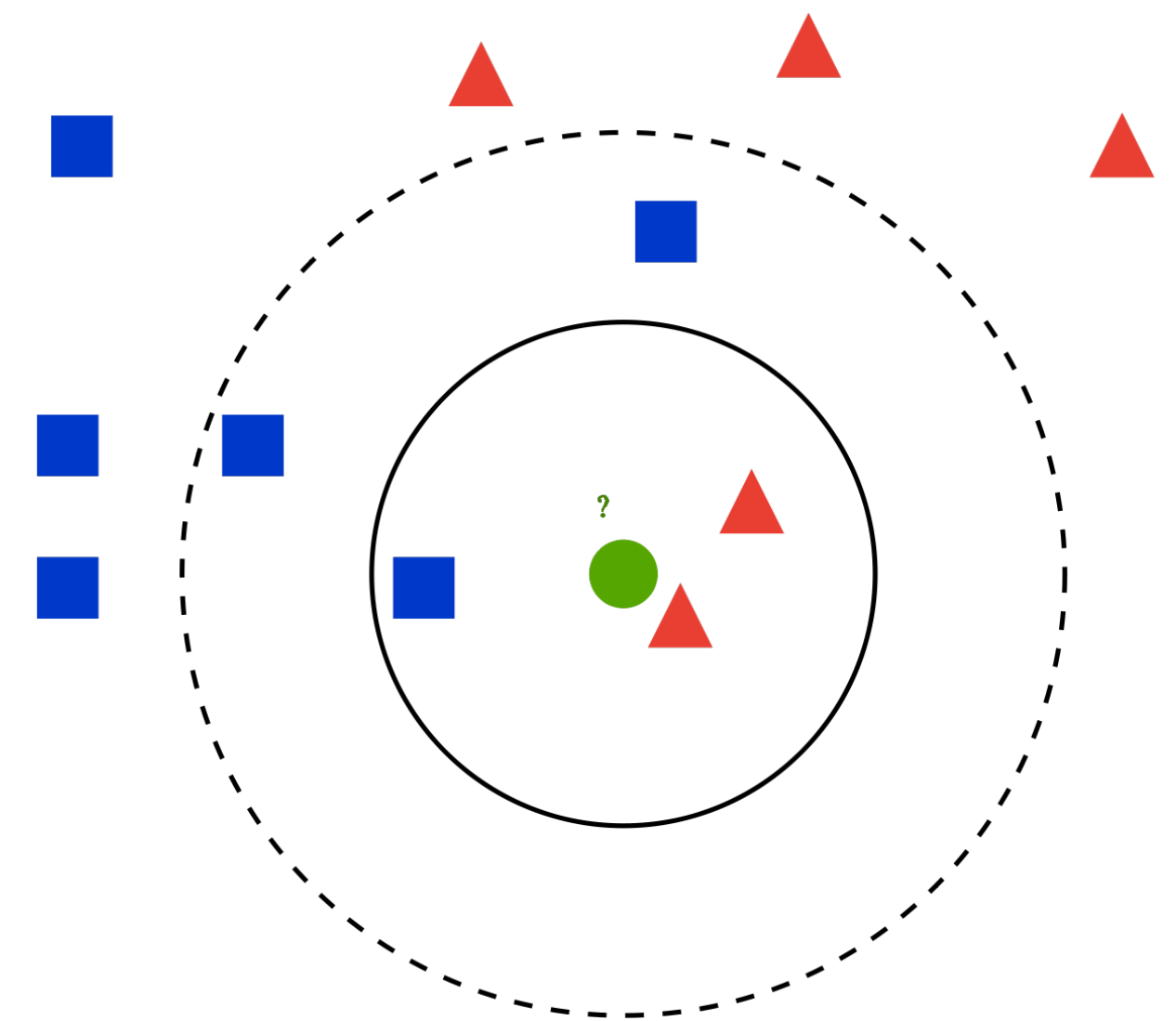
k-vizinhos

Algoritmo:

1. Identifique os k-vizinhos mais próximos do vetor de atributos **X** que se quer classificar.
2. Determine o número de vizinhos em cada classe.
3. Classifique **X** como pertencente à classe que resultou em um maior número de vizinhos (a moda entre o número de classes).

$$p(y = j | \mathbf{x}) = \frac{1}{k} \sum_{i \in R} \mathbb{I}\{y_i = j\}$$

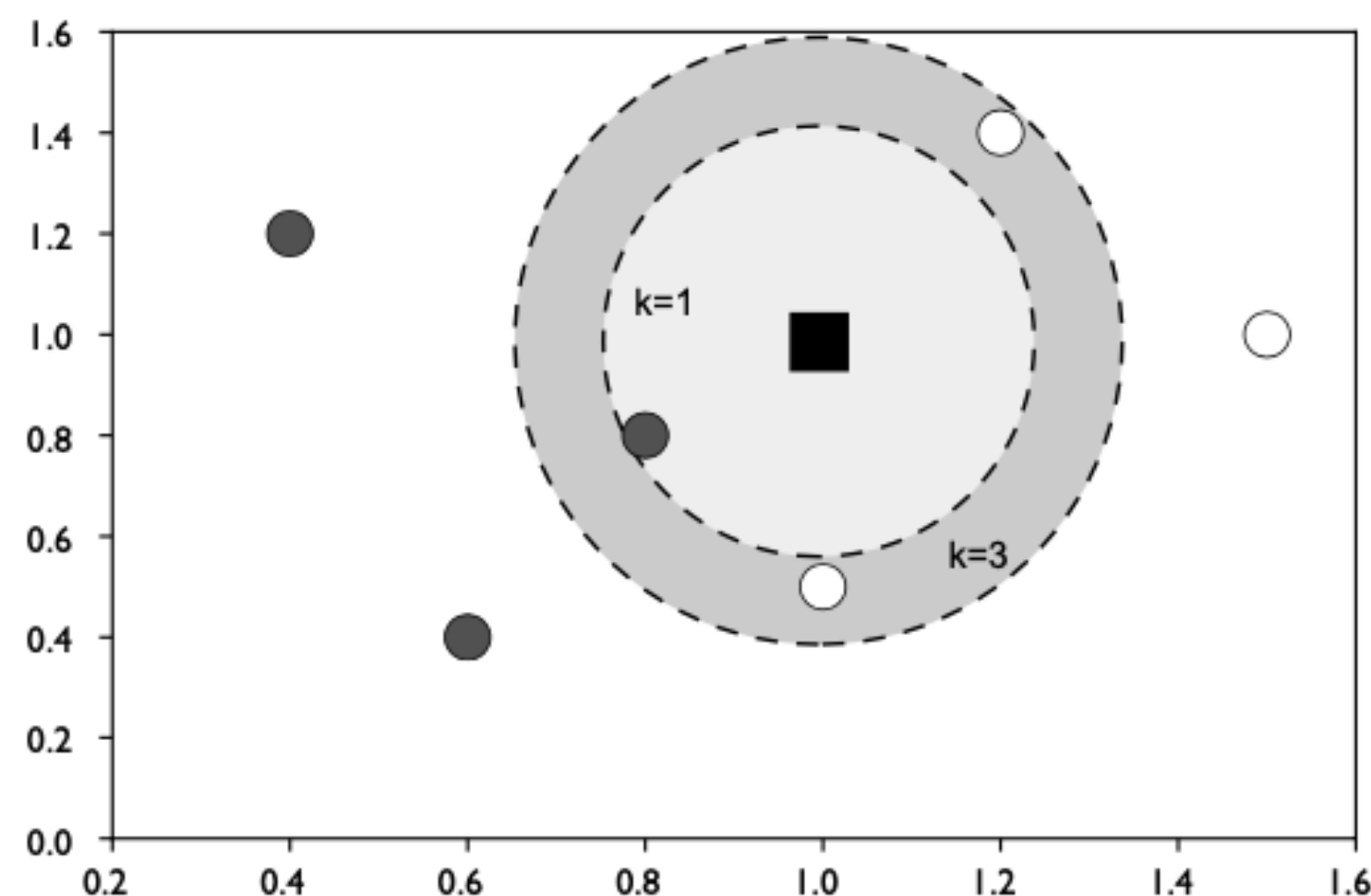
$$D(X, Y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$



k-vizinhos

- Exemplo:

Observação: $\mathbf{x} = [1,1]^T$



$k = 1$

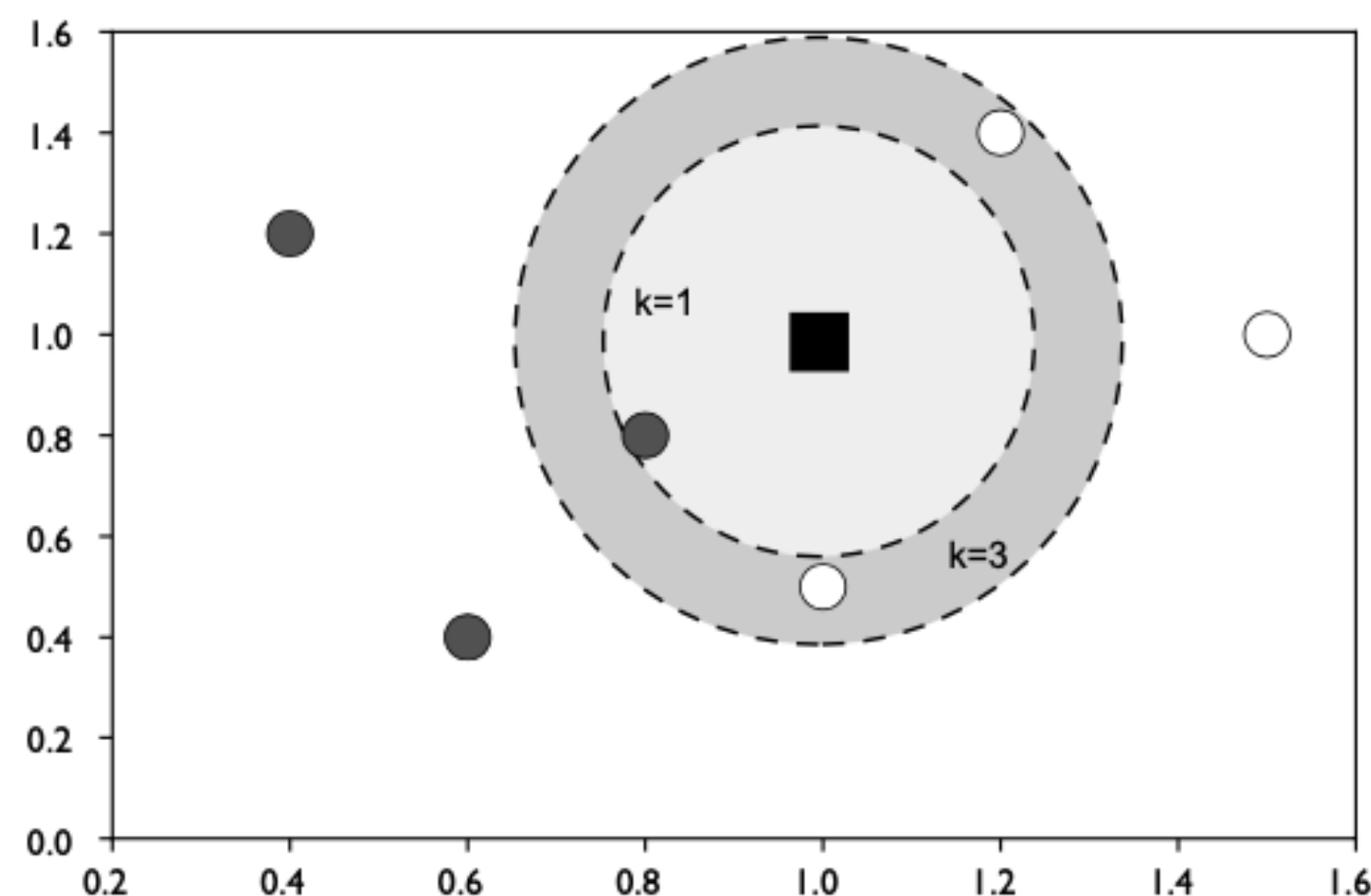
```
1 from scipy.spatial import distance
2 import statistics
3 import matplotlib.pyplot as plt
4
5 def knn(x_train, y_train, x_test, k):
6     distances = [] #lista que armazena as distâncias
7     x1 = x_test # elemento a ser classificado
8     for x2 in x_train: # distancia entre a observacao elementos no conjunto de teste
9         dist = distance.euclidean(x1,x2)
10        distances.append(dist)
11    indices = []
12    cl = []
13    for i in range(0,k):
14        ind = np.argmin(distances) #elemento no conjunto de teste mais proximo
15        #print('distance:', distances[ind], 'index:', ind, 'class:', y_train[ind])
16        distances[ind] = np.max(distances)
17        indices.append(ind)
18        cl.append(y_train[ind]) #guarda a classe
19    print("Classes:", cl)
20    classification = statistics.mode(cl) # encontra a classe
21    return classification
22
23 k=1 # numero de vizinhos
24 x_train = np.array([[1,0.5],[0.8,0.8],[1.2,1.4],[0.6,0.4],[0.4,1.2],[1.5,1]])
25 y_train = np.array(['white','gray','white','gray','gray','white'], dtype = 'str')
26 x_test = np.array([1,1])
27 # realiza a classificacao
28 cl = knn(x_train, y_train, x_test, k)
29 print("Classification:", cl)
30 # mostra os dados
31 plt.scatter(x_train[:,0],x_train[:,1],c=y_train, s=150, marker='o', edgecolor='black')
32 plt.plot(x_test[0],x_test[1], marker='s', markersize=15, color="black")
33 plt.xlim(0.2,1.6)
34 plt.ylim(0,1.6)
35 plt.savefig('knn.eps')
36
37 plt.show(True)
```

Classes: ['gray']
Classification: gray

k-vizinhos

- Exemplo:

Observação: $\mathbf{x} = [1,1]^T$



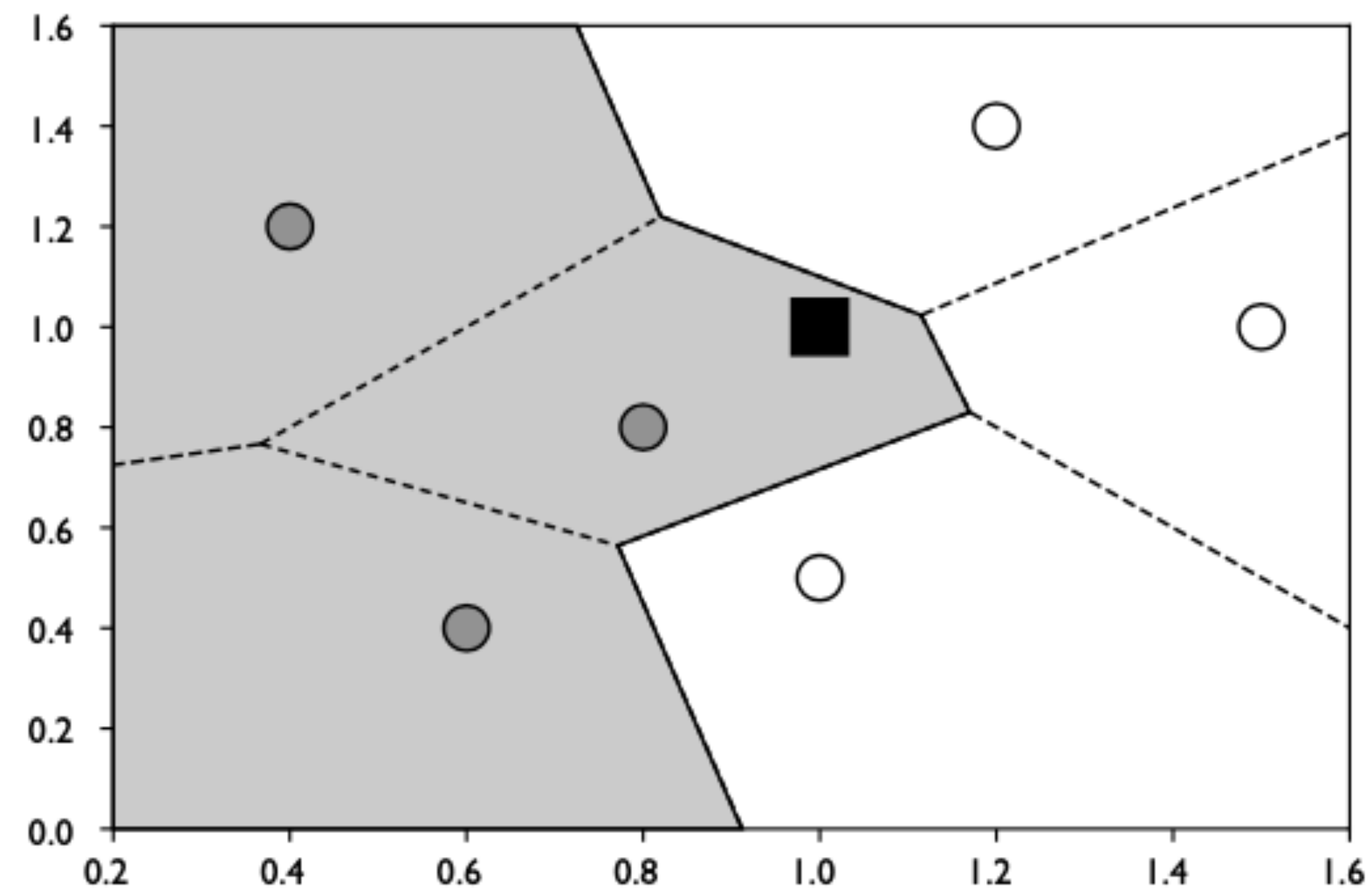
$k = 3$

```
1 from scipy.spatial import distance
2 import statistics
3 import matplotlib.pyplot as plt
4
5 def knn(x_train, y_train, x_test, k):
6     distances = [] #lista que armazena as distâncias
7     x1 = x_test # elemento a ser classificado
8     for x2 in x_train: # distancia entre a observacao elementos no conjunto de teste
9         dist = distance.euclidean(x1,x2)
10        distances.append(dist)
11    indices = []
12    cl = []
13    for i in range(0,k):
14        ind = np.argmin(distances) #elemento no conjunto de teste mais proximo
15        #print('distance:', distances[ind], 'index:', ind, 'class:', y_train[ind])
16        distances[ind] = np.max(distances)
17        indices.append(ind)
18        cl.append(y_train[ind]) #guarda a classe
19    print("Classes:", cl)
20    classification = statistics.mode(cl) # encontra a classe
21    return classification
22
23 k=3 # numero de vizinhos
24 x_train = np.array([[1,0.5],[0.8,0.8],[1.2,1.4],[0.6,0.4],[0.4,1.2],[1.5,1]])
25 y_train = np.array(['white','gray','white','gray','gray','white'], dtype = 'str')
26 x_test = np.array([1,1])
27 # realiza a classificacao
28 cl = knn(x_train, y_train, x_test, k)
29 print("Classification:", cl)
30 # mostra os dados
31 plt.scatter(x_train[:,0],x_train[:,1],c=y_train, s=150, marker='o', edgecolor='black')
32 plt.plot(x_test[0],x_test[1], marker='s', markersize=15, color="black")
33 plt.xlim(0.2,1.6)
34 plt.ylim(0,1.6)
35 plt.savefig('knn.eps')
36
37 plt.show(True)
```

Classes: ['gray', 'white', 'white']
Classification: white

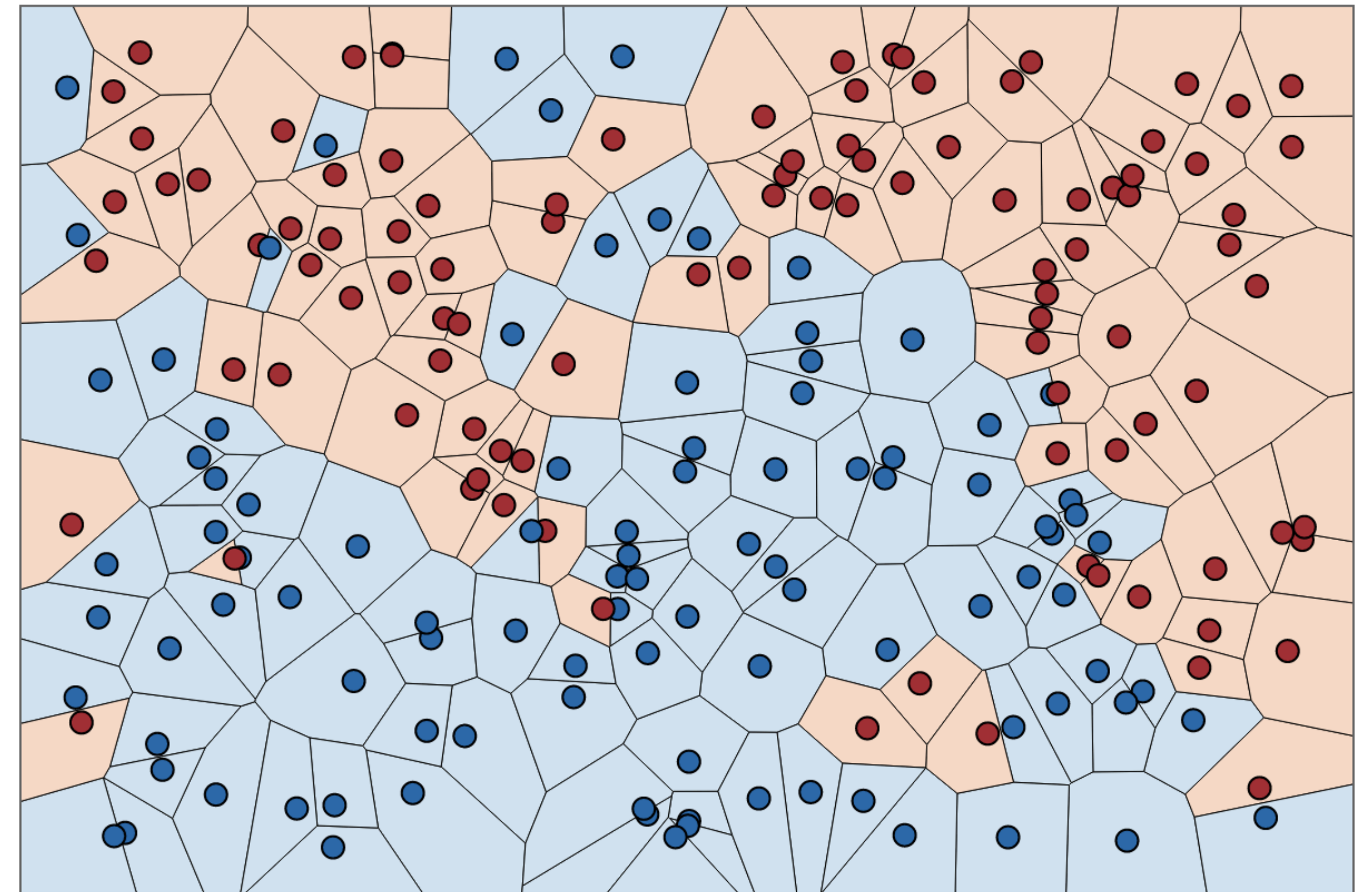
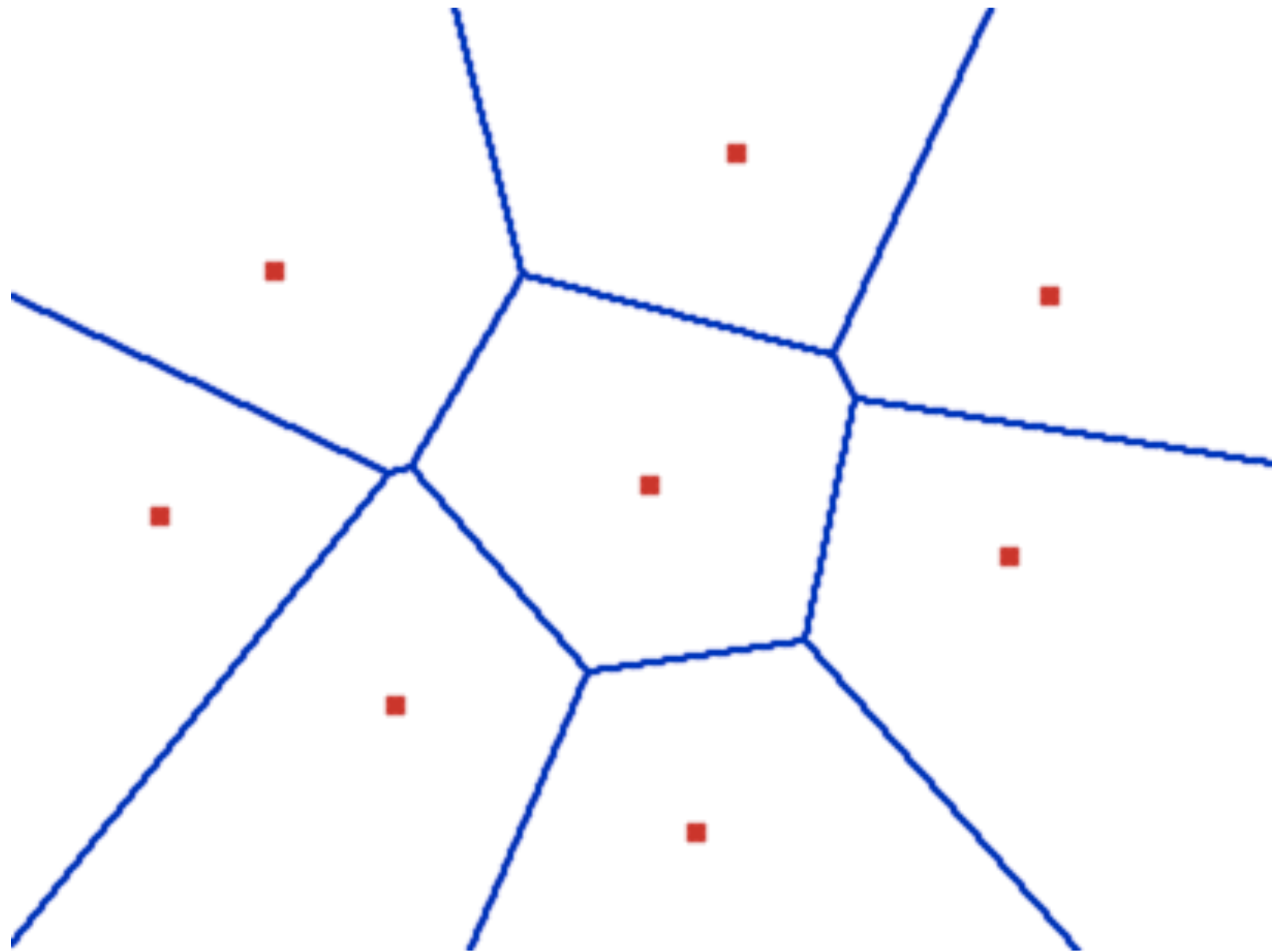
k-vizinhos

- A região de decisão forma telhas de Voronoi:



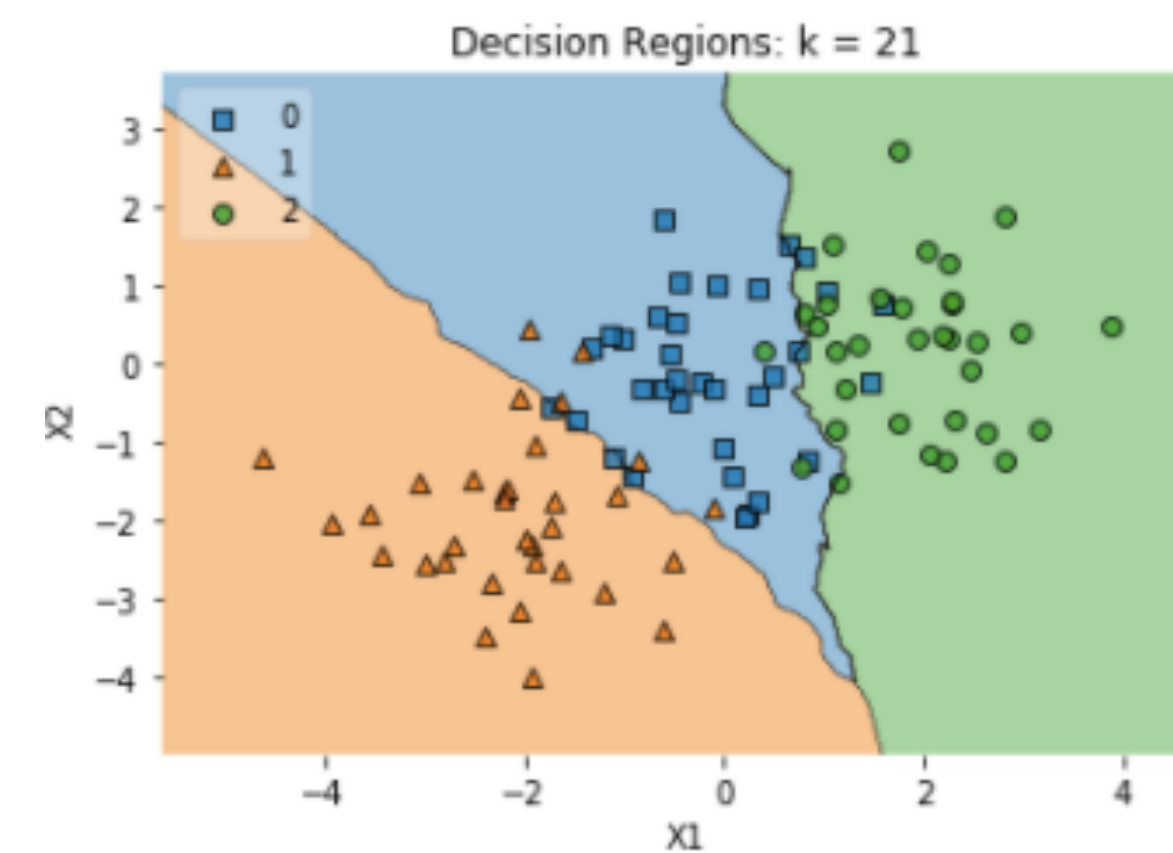
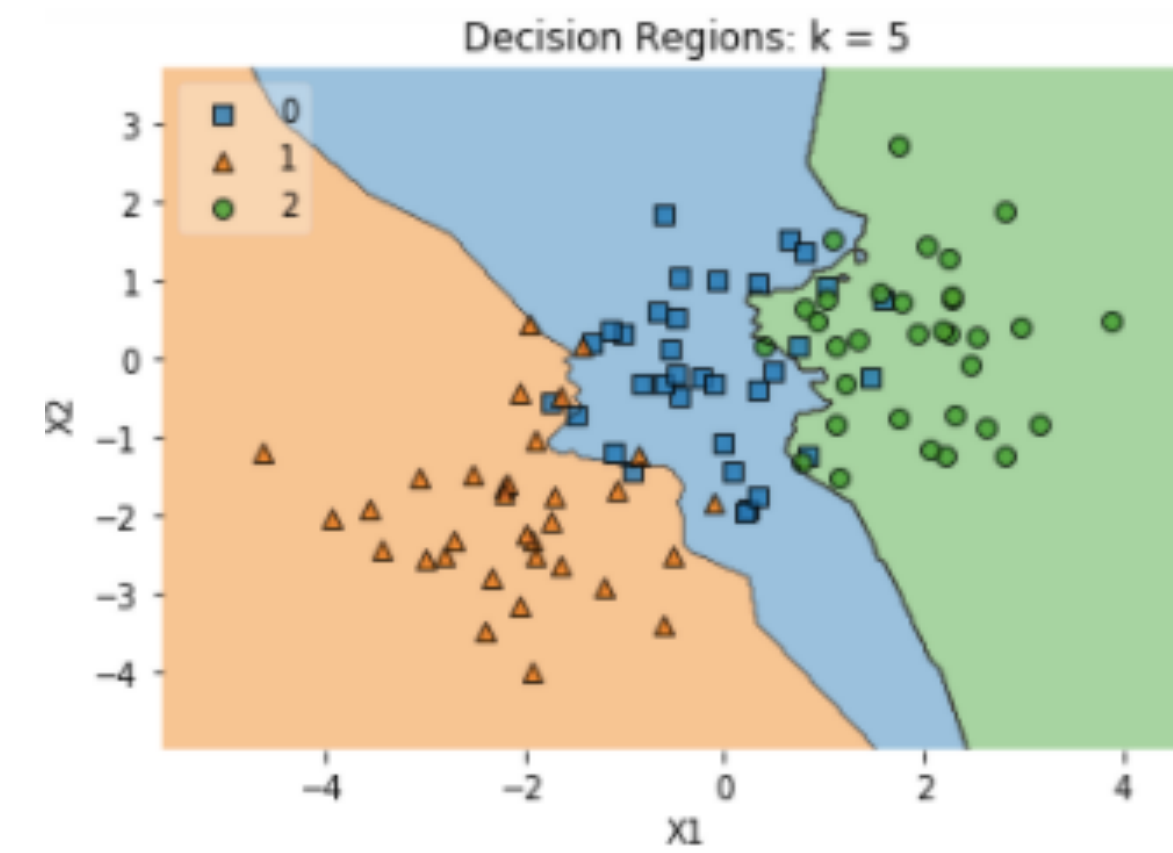
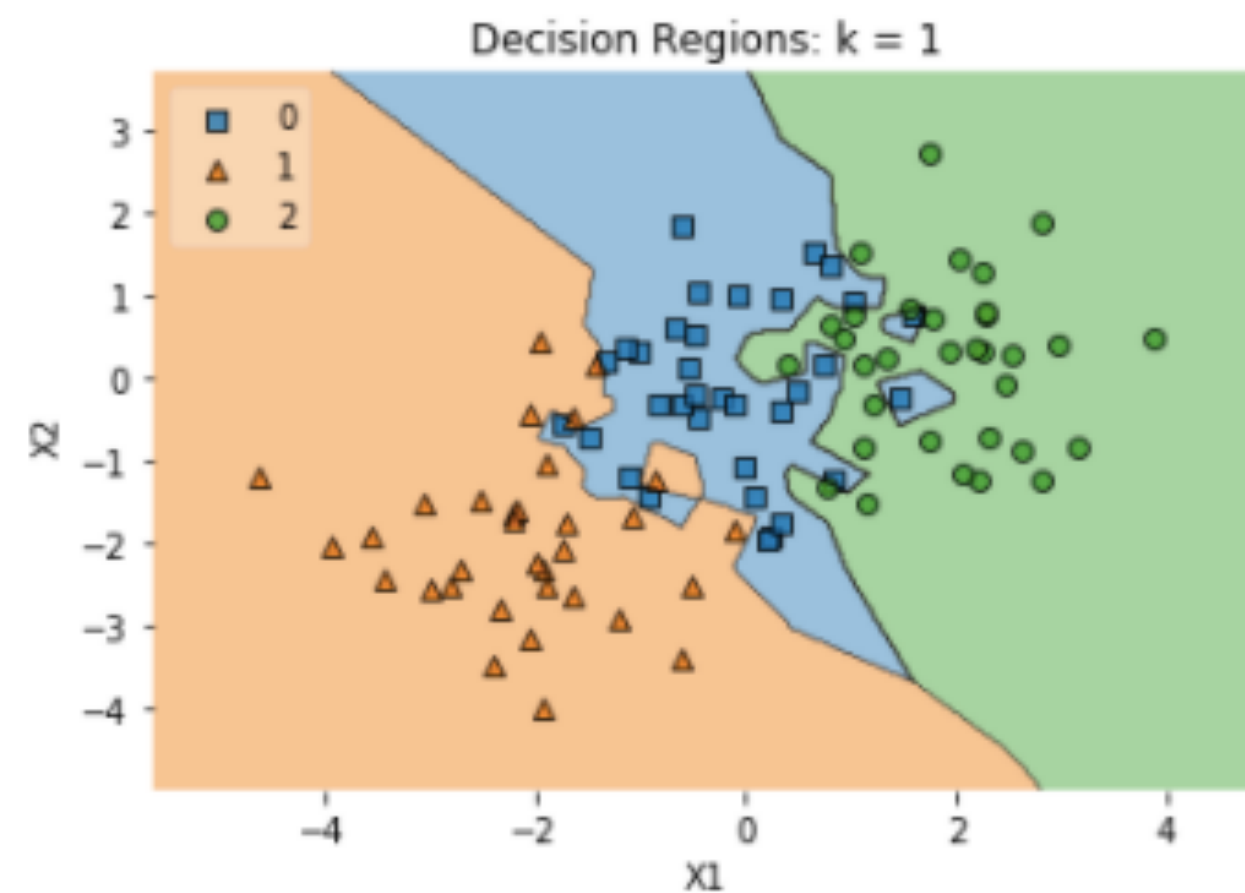
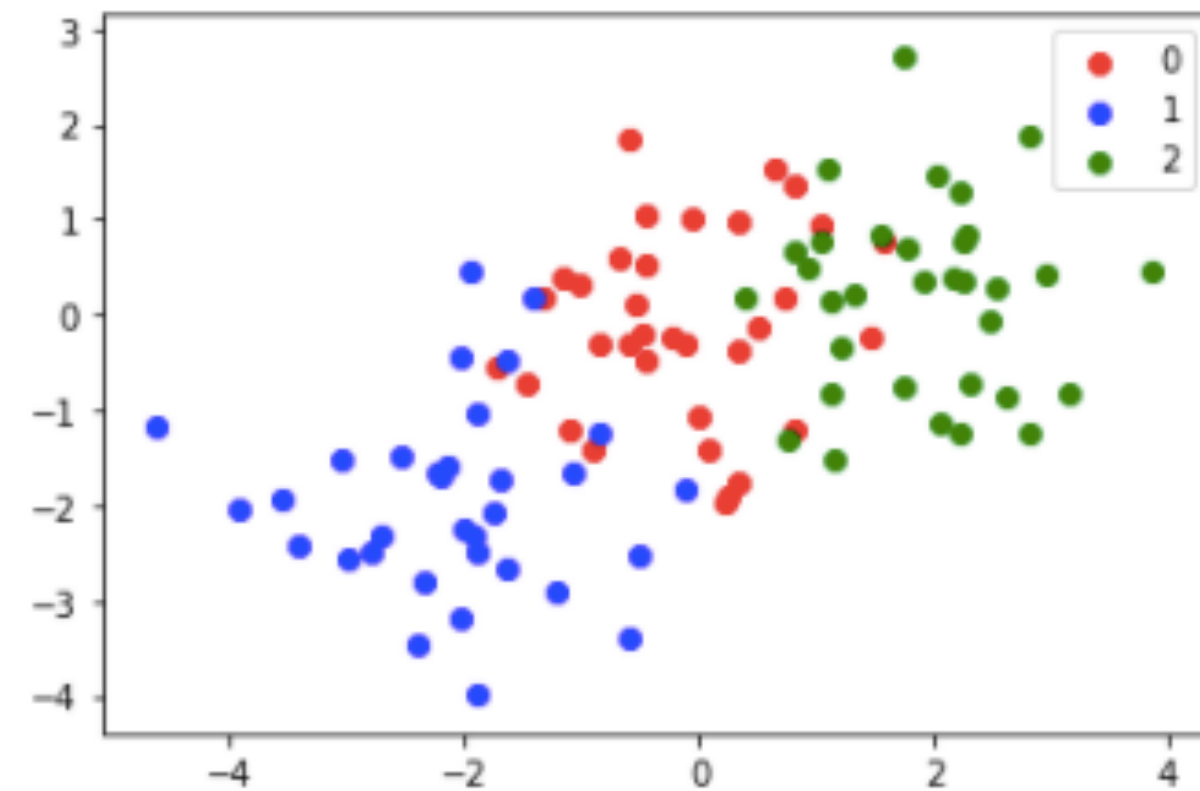
k-vizinhos

- Regiões de separação formam telhas de Voronoi.



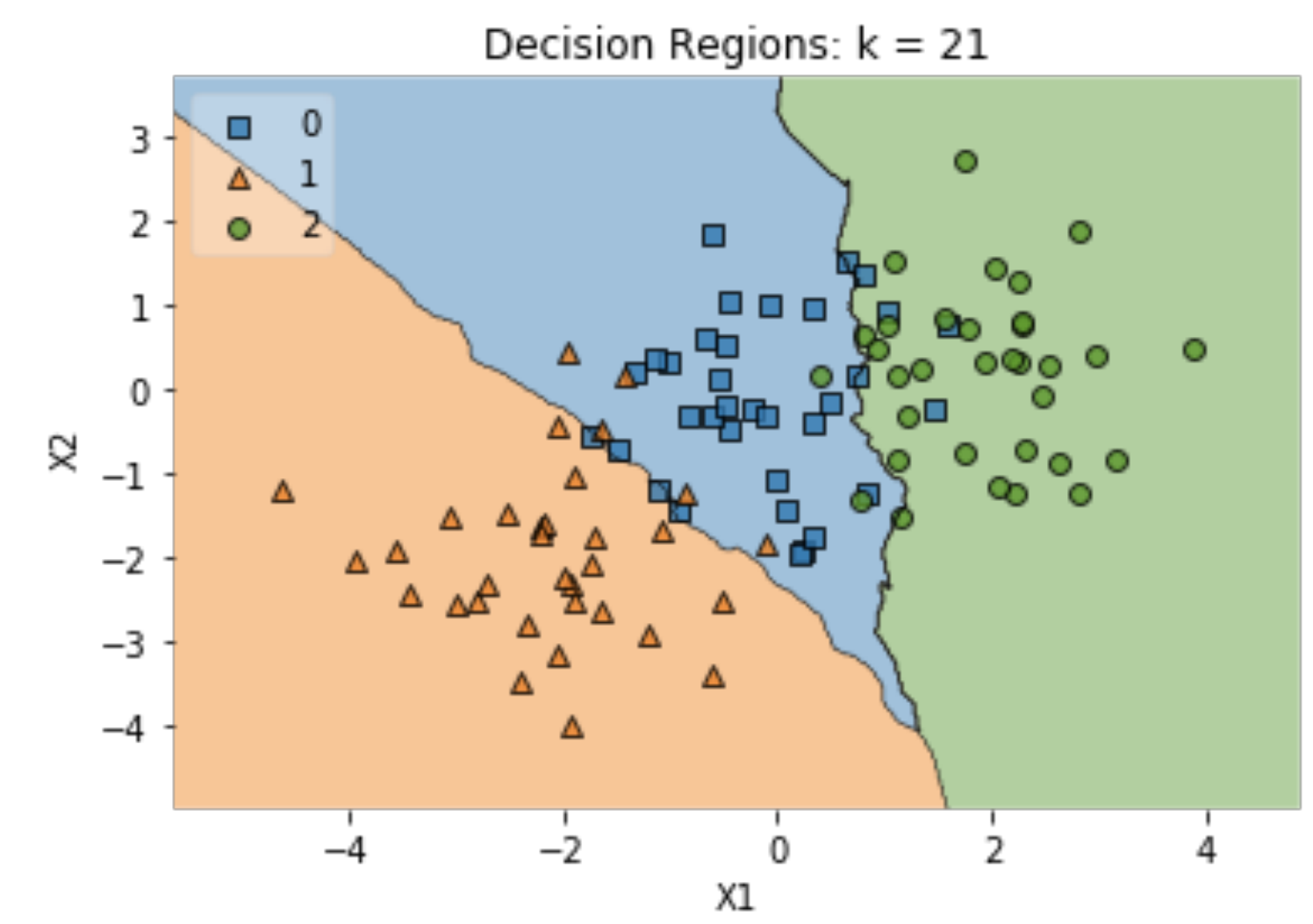
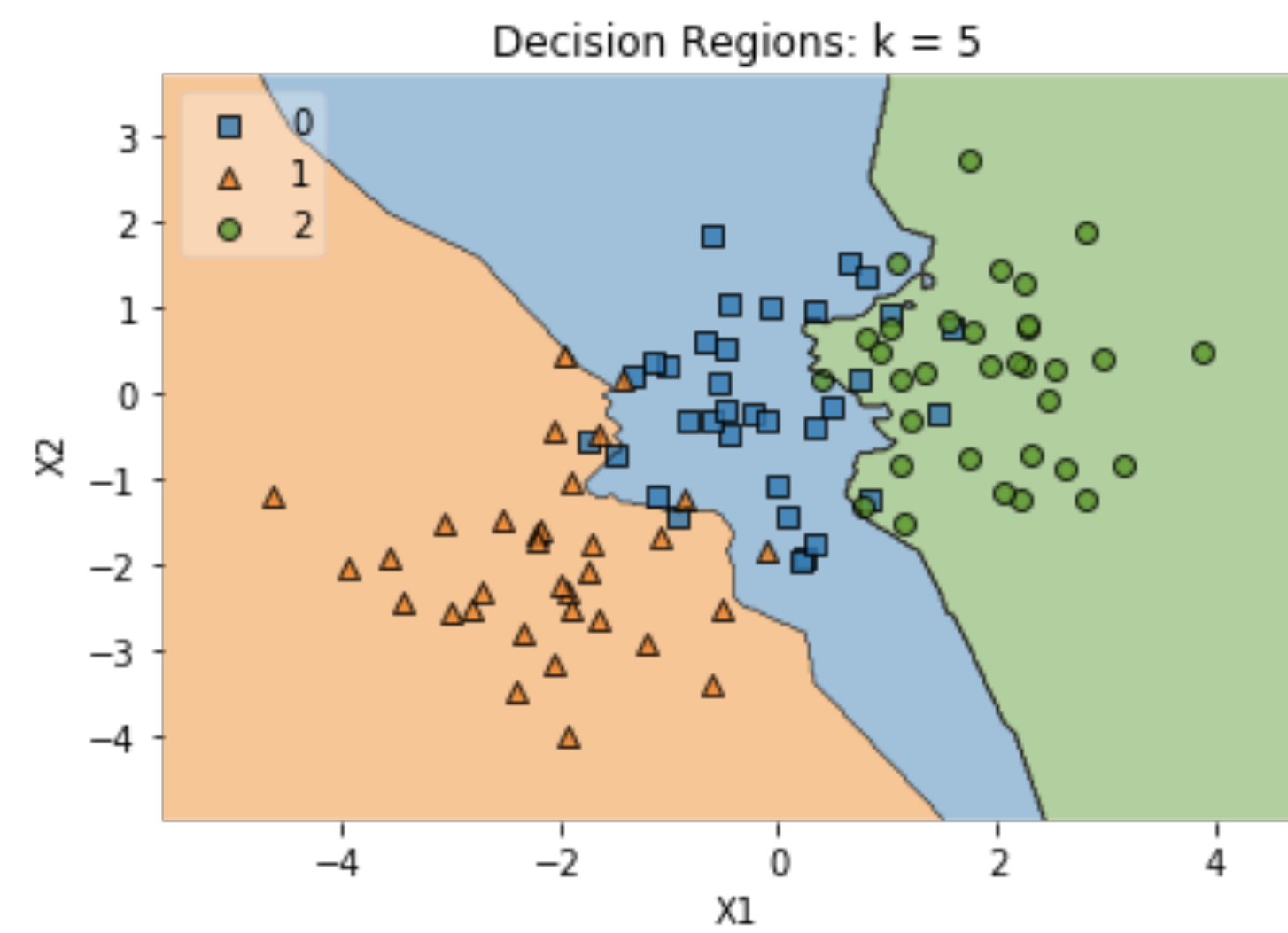
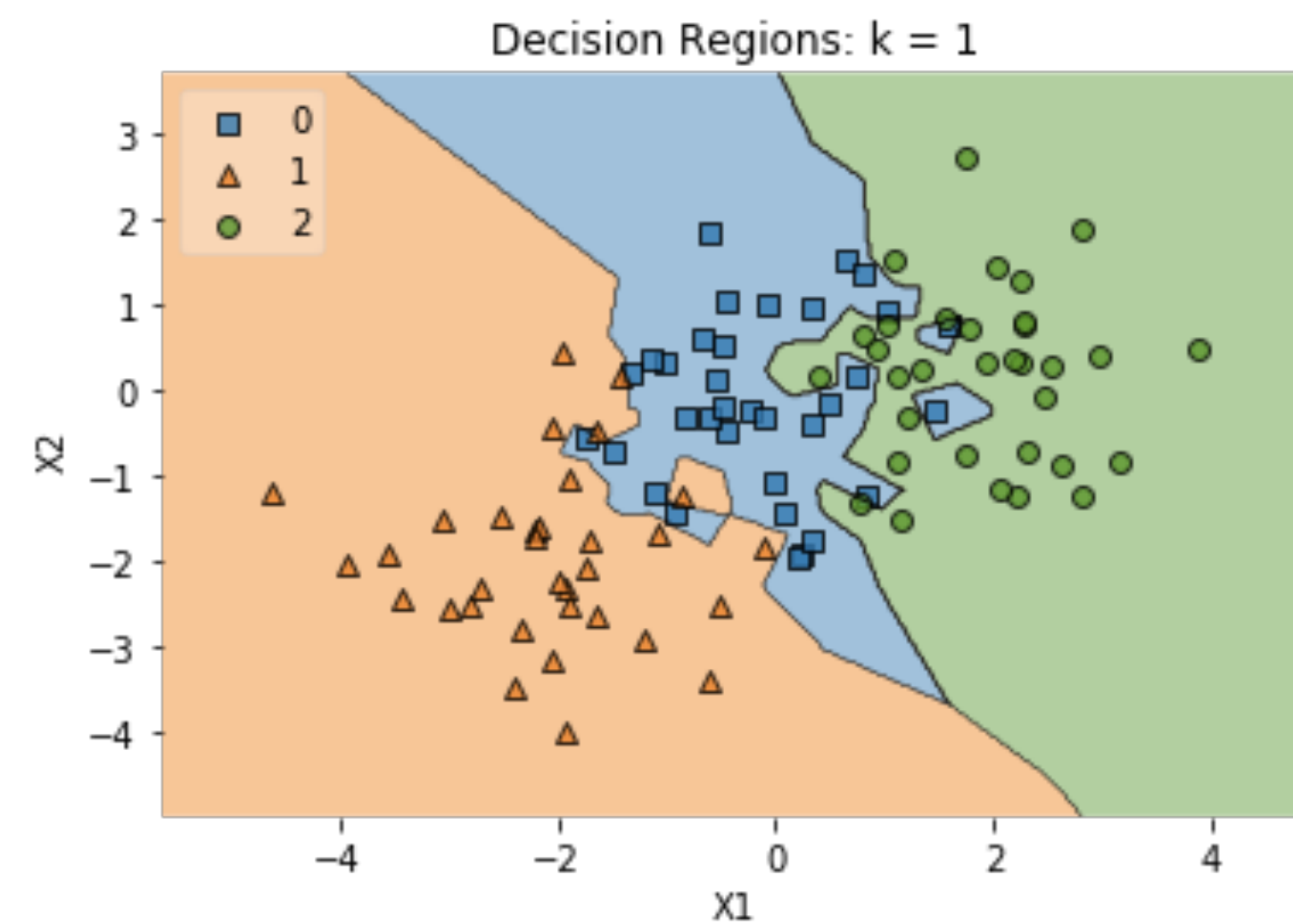
k-vizinhos

- Notem que a região de decisão depende de k .



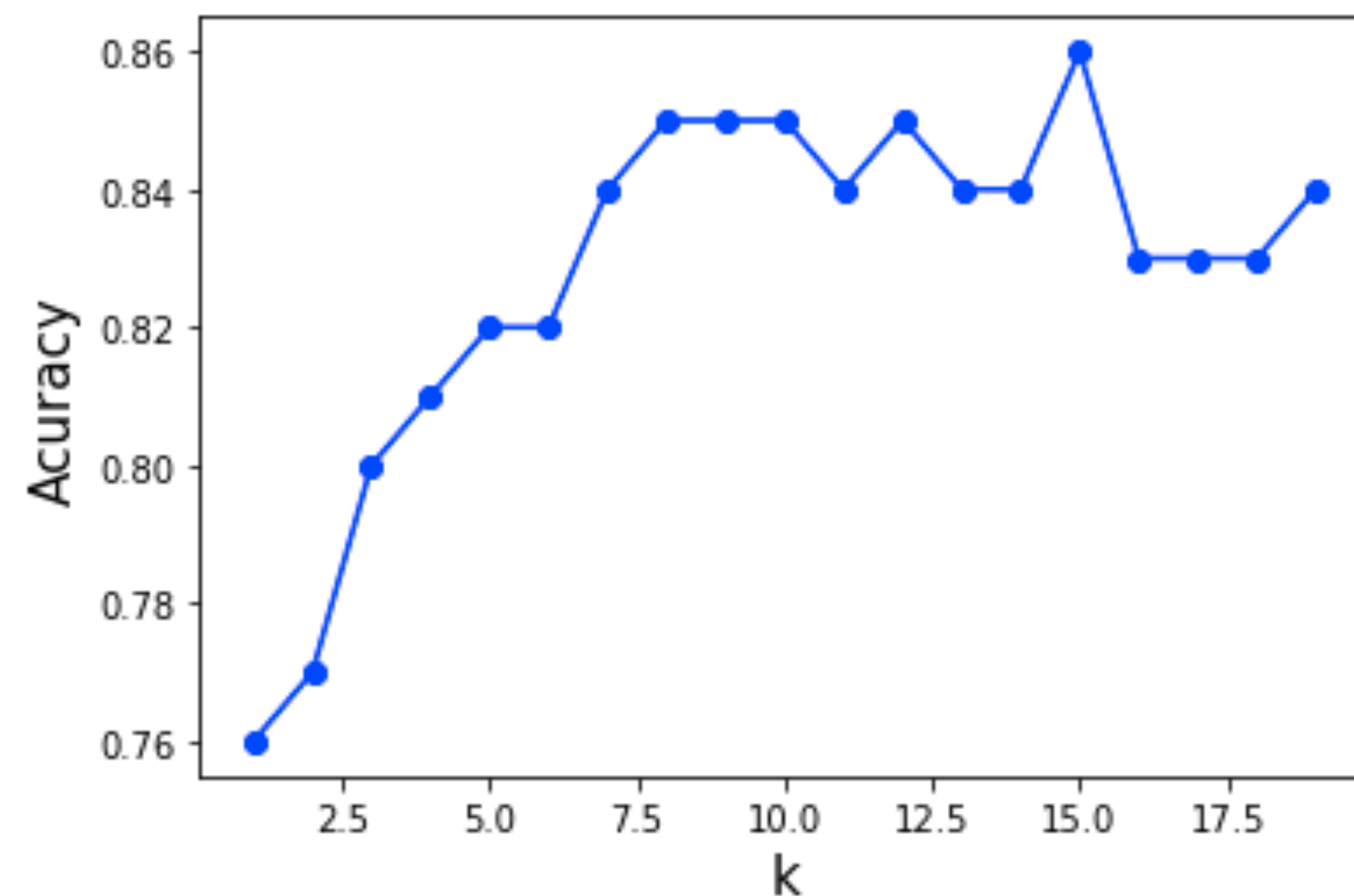
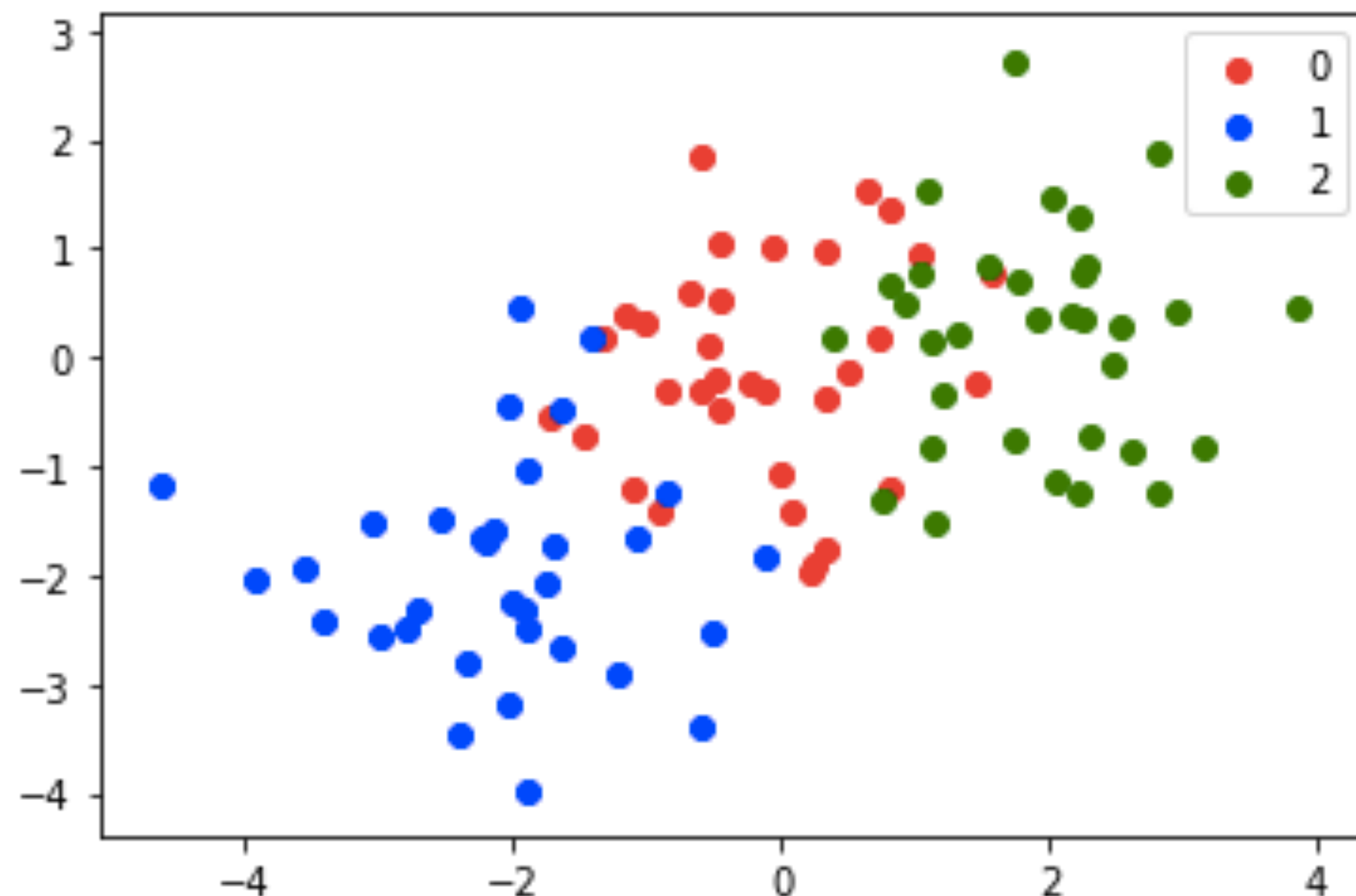
k-vizinhos

- Para $k=1$, vemos que a região de decisão se “ajusta muito” ao dados.
- Para k muito grande pode ocorrer uma suavização das região de decisão.



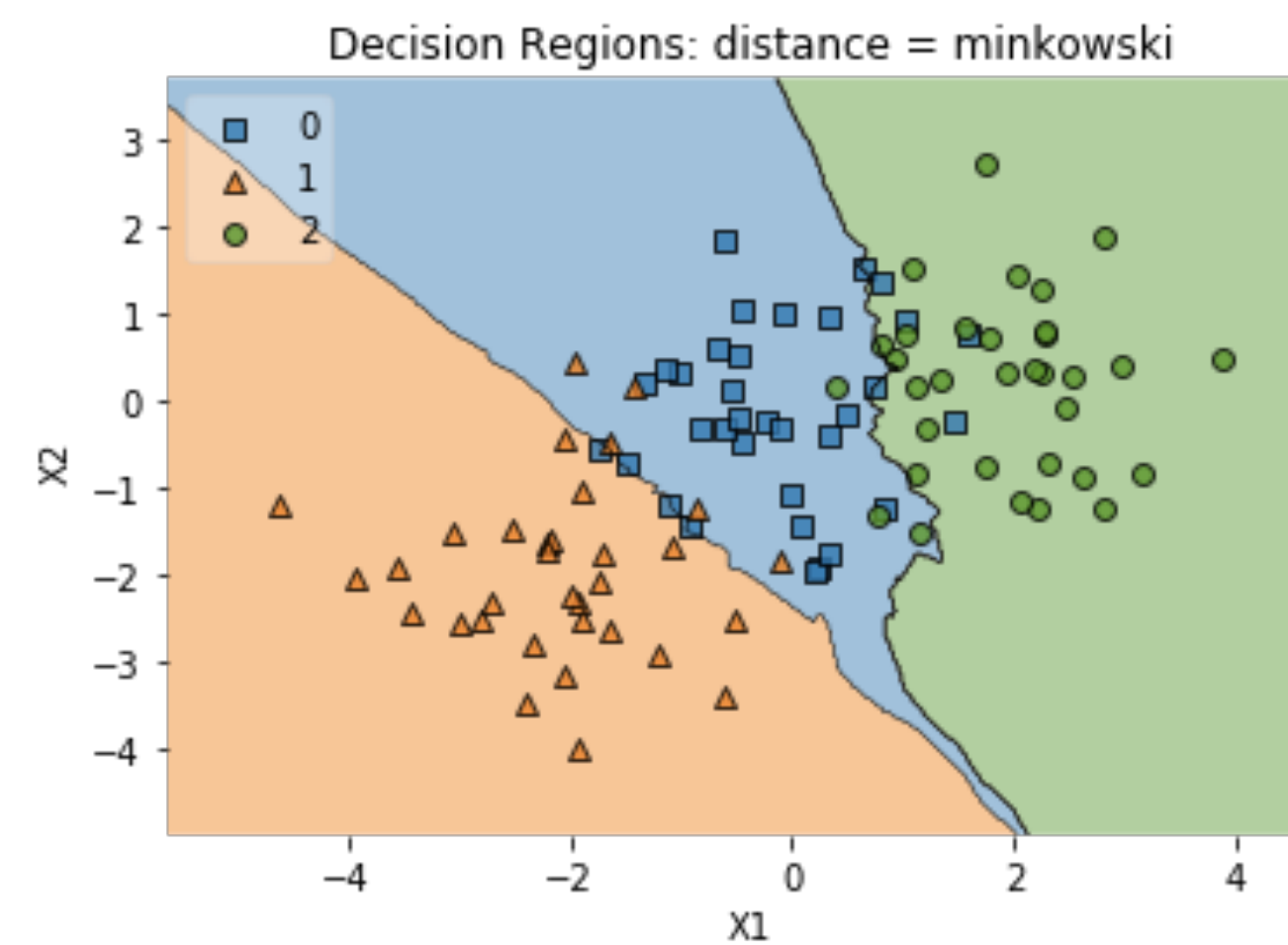
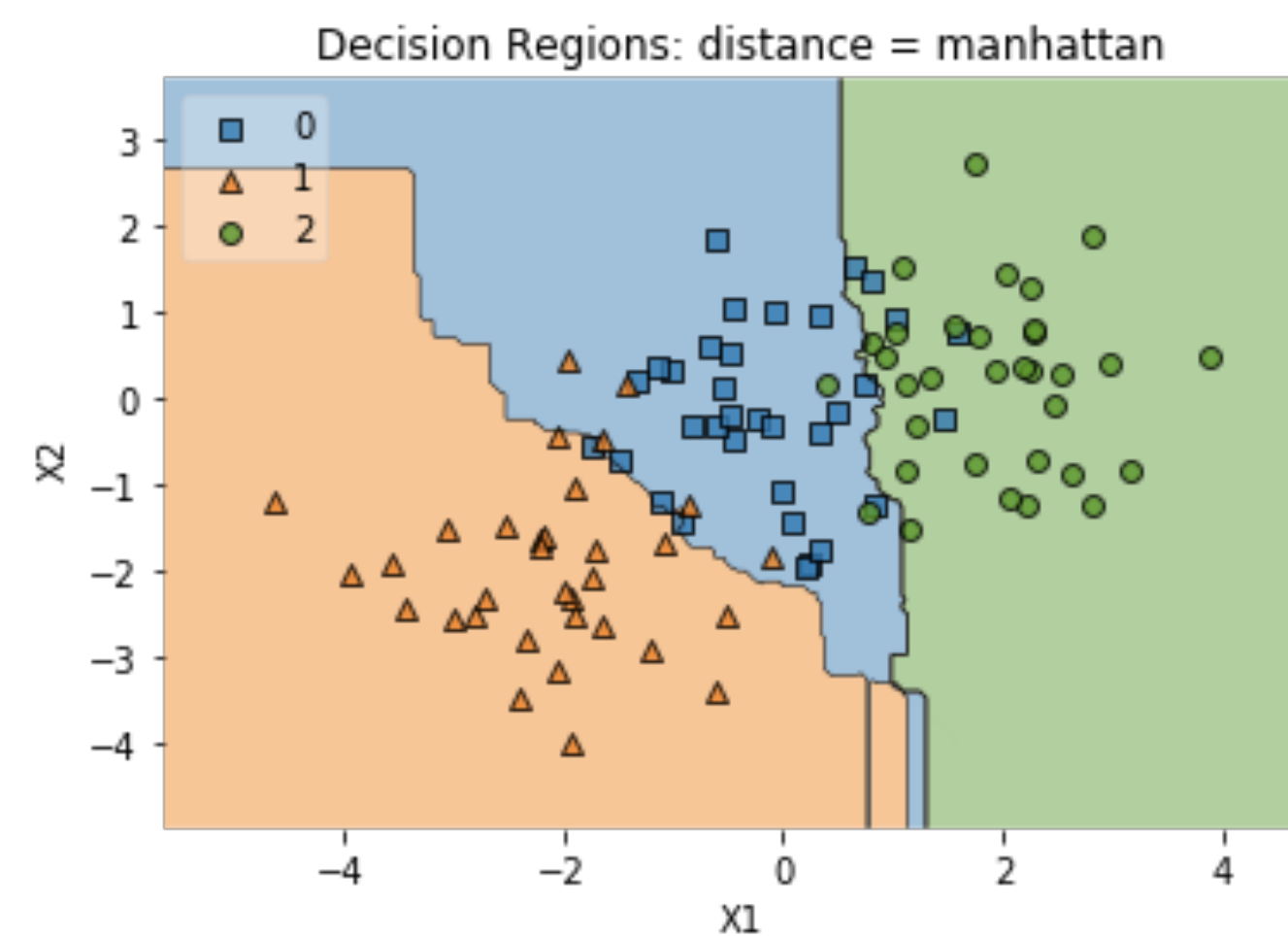
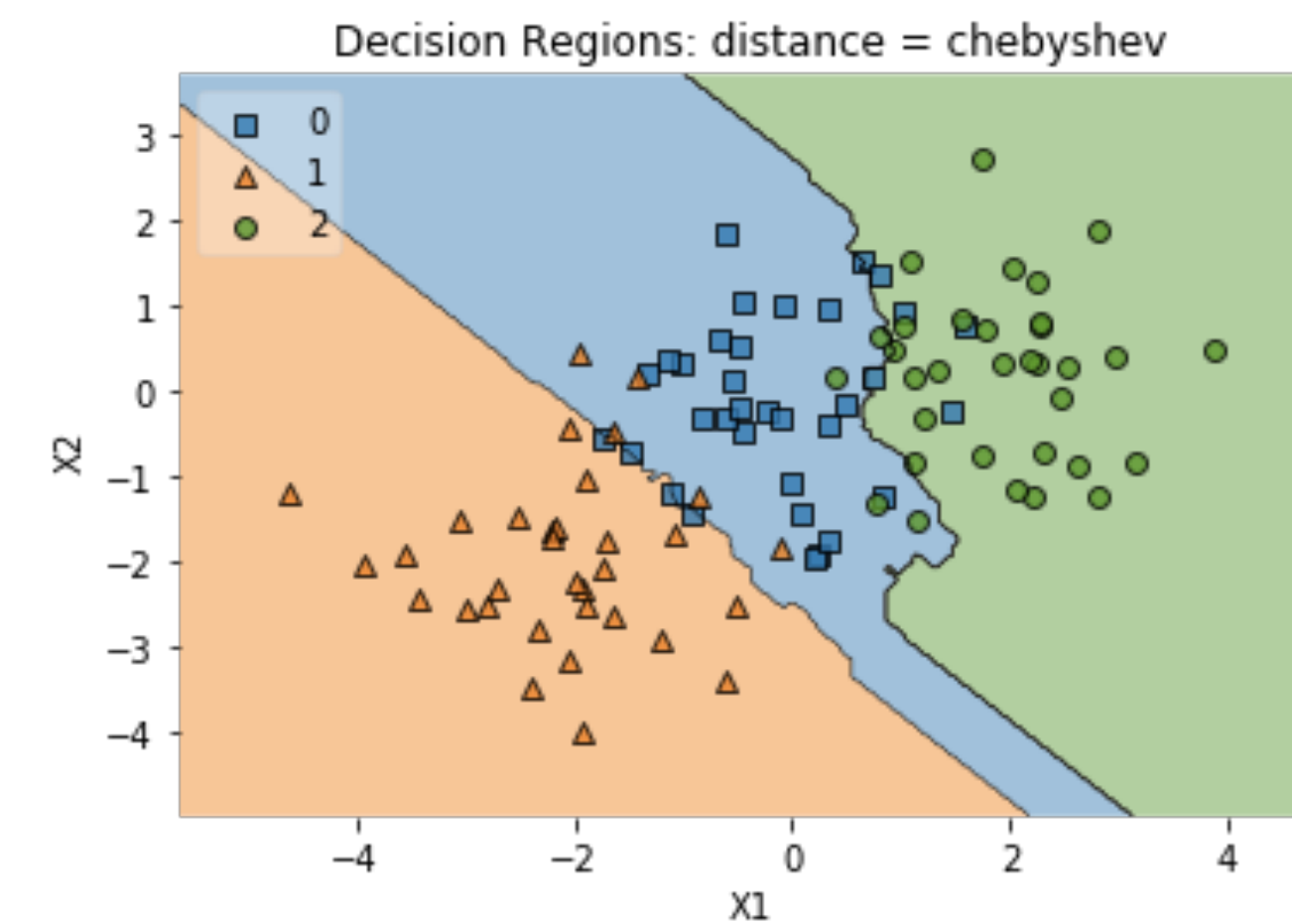
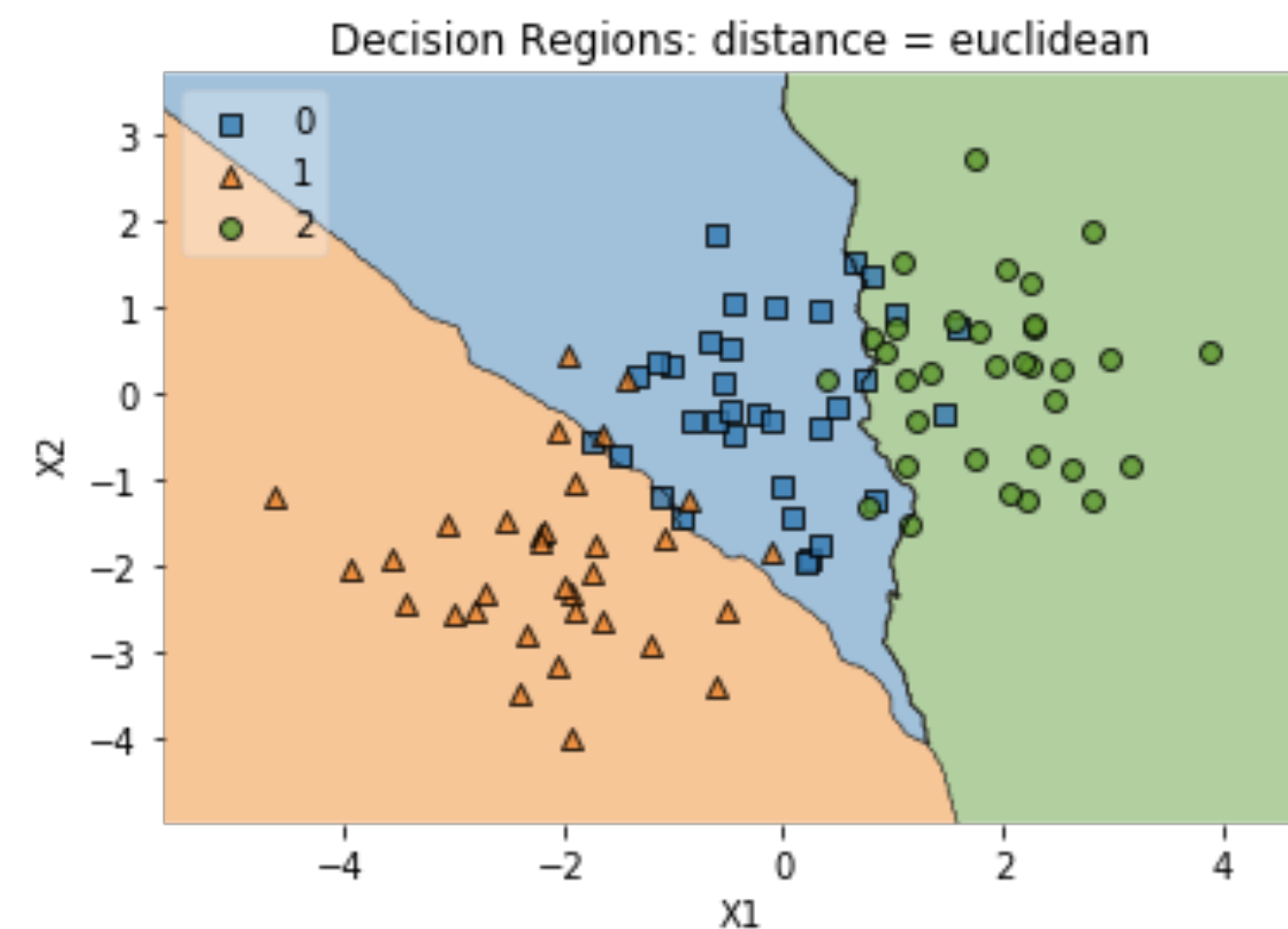
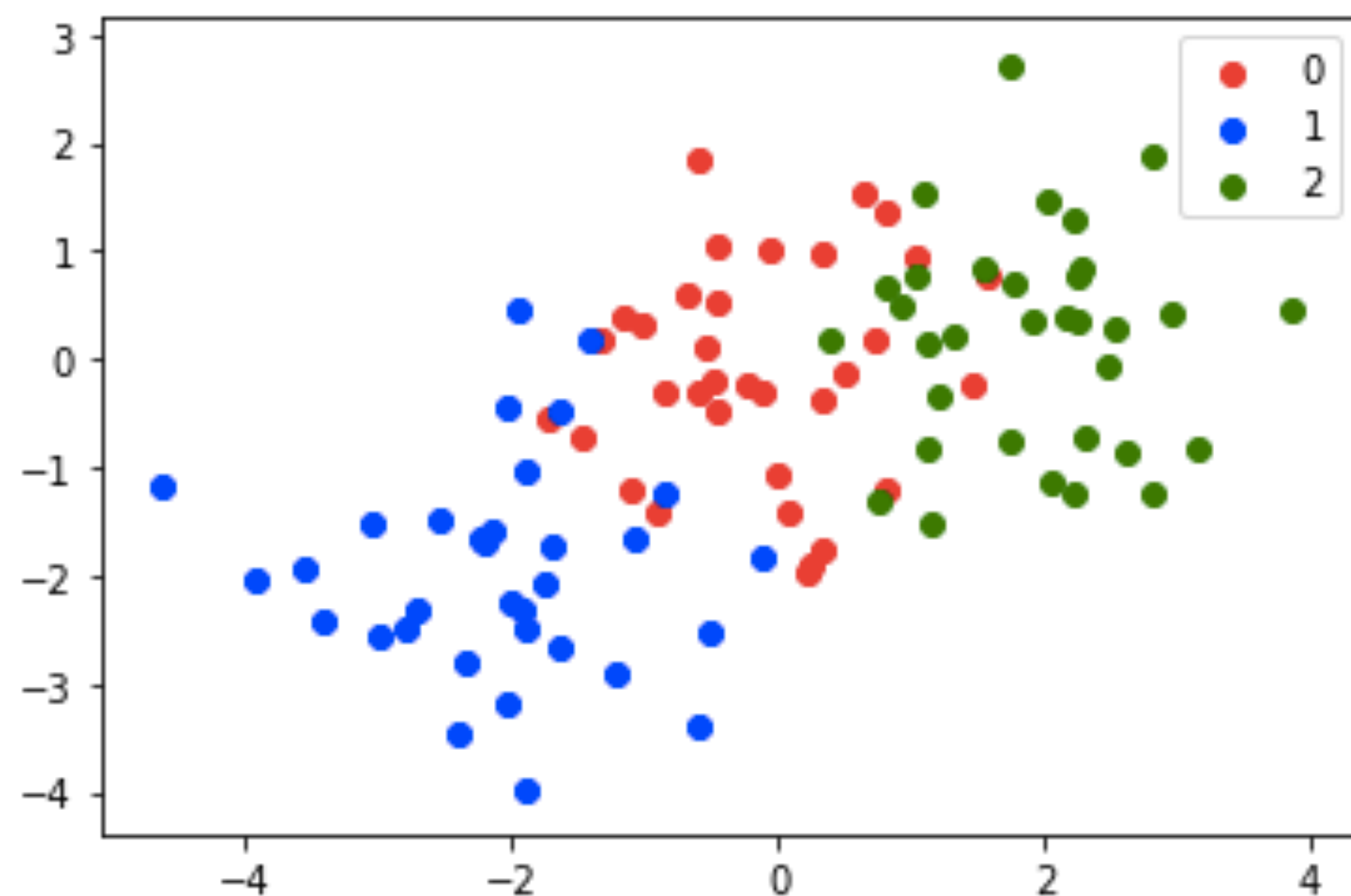
k-vizinhos

- Qual o melhor valor de k?
- A melhor maneira de encontrar o melhor valor de k é usar validação cruzada e uma medida para avaliar o resultado da classificação, como a acurácia.



k-vizinhos

- Qual métrica usar?



k-vizinhos

- Importante: Normalização e Padronização

$$Z_i = \frac{X_i - \min(X)}{\max(X) - \min(X)} \rightarrow Z_i \in [0,1]$$

$$Z_i = \frac{X_i - \mu_X}{\sigma_X} \rightarrow E[Z_i] = 0, V(Z_i) = 1$$

k-vizinhos

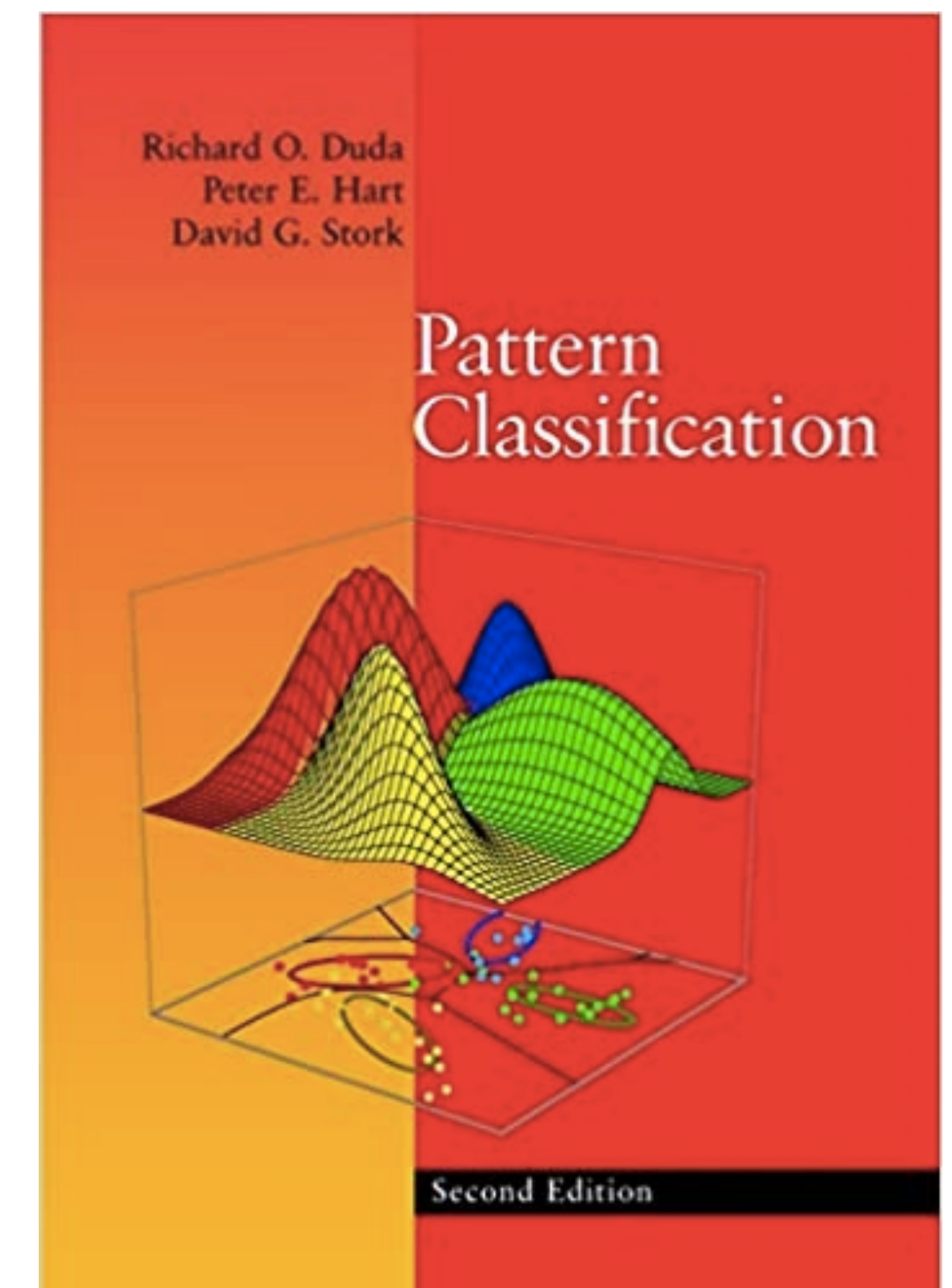
- **Propriedades:**
- O algoritmo não “aprende” um modelo, apenas memoriza objetos de treinamento.
- Adia computação para a fase de classificação.
- O algoritmo pode ser entendido como não paramétrico, dependendo apenas do número de vizinhos k .
- É um classificador não linear, não sendo restrito a regiões de separação lineares.
- Como geralmente a distância Euclidiana é considerada, é necessário normalizar ou padronizar os dados.

k-vizinhos

- Na década de 1960, o método foi retomado e formalizado por Thomas Cover e Peter Hart em um artigo seminal ("Nearest Neighbor Pattern Classification", IEEE Transactions on Information Theory, 1967).
- Eles provaram que o classificador 1-vizinho (1-NN) tem um erro assintótico no máximo duas vezes o erro do classificador de Bayes, considerado ótimo.
- Isso conferiu ao KNN uma base teórica sólida, mostrando que métodos simples poderiam ser estatisticamente consistentes.

$$p(C_j | \mathbf{X}) = \frac{1}{k} \sum_{i \in R} \mathbb{I}\{y_i = C_j\}$$

$$0 \leq P_e \leq 2P_B$$



- O método KNN surgiu no contexto do desenvolvimento da teoria estatística da classificação nos anos 1950–1960.
- Em 1951, Evelyn Fix e Joseph Hodges publicaram um relatório técnico da RAND Corporation ("Discriminatory Analysis, Nonparametric Discrimination: Consistency Properties") no qual introduziram uma forma inicial de classificação baseada em vizinhança, considerada a origem do KNN.
- O objetivo era criar um método não paramétrico, que não assumisse nenhuma distribuição prévia para os dados. Em vez de construir modelos matemáticos complexos, o método se baseava diretamente nos exemplos observados.

- Durante os anos 1970 e 1980, o KNN foi aplicado em:
 - Reconhecimento de padrões (imagens, fala e escrita).
 - Estatística não paramétrica, como forma de estimar densidades de probabilidade local.
 - Ciência da computação em tarefas de classificação e recuperação de informação.

k-vizinhos

- Apesar da simplicidade, o KNN enfrenta alguns problemas:
 - Alta dimensionalidade: conforme o número de atributos cresce, a noção de “proximidade” perde significado (fenômeno da “maldição da dimensionalidade”).
 - Custo computacional: exige armazenar todos os dados de treinamento e calcular distâncias a cada nova classificação.
- Esses desafios motivaram:
 - Técnicas de redução de dimensionalidade (PCA, LDA).
 - Métodos de ponderação dos vizinhos e KNN adaptativo.

k-vizinhos

- A partir dos anos 1990–2000, com a consolidação do ML, o KNN passou a ser ensinado como um dos algoritmos básicos de classificação supervisionada.
- Tornou-se referência por sua:
 - Interpretação intuitiva (baseada em exemplos).
 - Facilidade de implementação.
 - Utilidade como baseline para comparar novos métodos.

- Hoje o KNN ainda é utilizado em:
 - Sistemas de recomendação (usuários semelhantes).
 - Detecção de anomalias.
 - Análise de séries temporais (versões adaptadas).
 - Ciências biológicas (classificação de genes, proteínas).
 - Como método de referência em competições e benchmarks de aprendizado de máquina.

k-vizinhos

- **KNN clássico (ou básico)**
 - Cada amostra nova é classificada pela maioria dos rótulos entre os seus k vizinhos mais próximos.
 - A distância mais comum é a euclidiana, mas outras métricas (Manhattan, Minkowski, cosseno) também são usadas.
 - Pontos mais próximos têm o mesmo peso que pontos mais distantes.

- **KNN ponderado (Weighted KNN, WKNN)**

- Introduz pesos que diminuem com a distância: vizinhos mais próximos têm maior influência.
- Reduz erros quando há vizinhos próximos de classes diferentes.
- Fórmulas típicas:

- $w_i = \frac{1}{d(x, x_i)}$
- $w_i = e^{-\alpha d(x, x_i)}$

- **4. KNN adaptativo**

- O valor de k não é fixo, mas varia conforme a densidade local dos dados.
- Em regiões densas: usa um k maior.
- Em regiões esparsas: usa um k menor.
- Isso evita enviesar a classificação em regiões de fronteira.

Leitura adicional

- Machine Learning - A First Course for Engineers and Scientists

<http://smlbook.org/>

